



4

64

9600 sp

عملي

محتوى مجاني غير مخصص للبيع التجاري

كلية الهندسة المعلوماتية

السنة الرابعة - الذكاء الصناعي

Training

عملي مشترك



RB Informatcs; 19/11/2025

الشبكات العصبونية

التدريب (Training)

يعني العملية التي تجد فيها الشبكة العصبية تلقائياً أفضل قيم للمعاملات (parameters)، وهي الأوزان (Weights) من بيانات التدريب و هذه المعاملات هي التي تحدد سلوك الشبكة.

المعاملات (Parameters)

هي الأرقام التي "يتعلمها" النموذج. في أبسط شبكة عصبية، هذه هي الأوزان (Weights) على سبيل المثال: في شبكة بسيطة مثل "البسيبترون (Perceptron)" الذي تحدثنا عنه في المحاضرة الثانية، يمكنك تعديل هذه المعاملات يدوياً بالنظر إلى جدول الحقيقة (Truth Table) كان هناك 3 معاملات فقط، مما يجعل الأمر ممكناً.

المشكلة الحقيقية: في الشبكات العصبية الحقيقية (خاصة في التعلم العميق) عدد هذه المعاملات يتراوح بين الآلاف إلى عشرات الملايين، من المستحيل تماماً ضبط هذه المعاملات يدوياً.

الحل:

(التدريب) نترك الشبكة "تتعلم" هذه القيم بنفسها من البيانات. ولكن كيف تعرف الشبكة أنها أصبحت "أفضل"؟ هنا يأتي دور دالة الخسارة

ما هو الدليل الذي نتبعه أثناء التدريب؟

لا يمكن للشبكة أن "تخمن" الأوزان الصحيحة. تحتاج إلى "دليل" يخبرها إذا كانت تقترب من الإجابة الصحيحة أو تبتعد عنها. هذا الدليل هو دالة الخسارة (Loss Function).

■ دالة الخسارة: هي دالة رياضية تأخذ مخرجات الشبكة والمخرجات الصحيحة (Labels)، وتُخرج رقماً واحداً. هذا الرقم يمثل "مدى سوء" أداء الشبكة الحالي.

■ الهدف: هدف التدريب هو تعديل الأوزان بشكل مستمر لتقليل قيمة دالة الخسارة إلى أقل ما يمكن. عندما تكون الخسارة عند أقل قيمة لها، يكون أداء الشبكة هو الأفضل.

حل مشاكل التعرف على الأنماط (مثل التعرف على الأرقام المكتوبة بخط اليد)، وهو نهج التعلم الآلي التقليدي. هذا النهج يمثل خطوة متقدمة عن محاولة كتابة الخوارزميات يدوياً، ولكنه لا يزال يعتمد على تدخل بشري حاسم، وهو ما يميزه عن التعلم العميق الذي جاء لاحقاً.



1. المشكلة والحل المقترح : استخدام البيانات بفعالية :

المشكلة :

محاولة كتابة خوارزمية من الصفر للتعرف على الرقم "5" أمر شبه مستحيل. لماذا؟ لأن كل شخص يكتب "5" بطريقة مختلفة: بعضهم يكتبه بحلقة صغيرة، وبعضهم يكتبه بخط مائل، وآخرون يكتبونه بشكل غير متقن. وضع قواعد صارمة للبرنامج لتمييز كل هذه الحالات مهمة شاقة وغير عملية.

الحل :

بدلاً من أن نخبر الحاسوب "كيف" يتعرف على "5"، نعطيه "أمثلة" ونتركه هو "يتعلم" كيف. الطريقة لفعل ذلك هي عبر خطوتين:

- استخلاص الميزات feature extraction
- تعلم الآلة للنمط machine learning

2. ما هي الميزة (feature) :

محول (converter) مصمم لاستخلاص البيانات الجوهرية (المهمة) من البيانات الأولية (الصورة) بدقة مثال: تخيل أنك تصف سيارة لشخص لم يرها من قبل. بدلاً من إعطائه صورة فوتوغرافية (البيانات الأولية)، يمكنك إعطاؤه قائمة من الصفات :

اللون: أحمر.

النوع: مارسيدس.

عدد الأبواب: 4.

هذه القائمة [مارسيدس , 4 , احمر] هي متجه الميزات (تحويل الصورة إلى مجموعة من الأرقام والتصنيفات التي تصف جوهرها).

في مجال رؤية الحاسوب، الميزات هي خوارزميات رياضية مصممة لاستخراج خصائص معينة من الصورة، من أشهر الخوارزميات المستخدمة لاستخلاص هذه الميزات: SIFT , SURF , HOG

3. كيف يعمل النهج كاملاً:

■ المدخل :
صورة للرقم 5

■ استخلاص الميزات (بشري) :
نطبق خوارزمية مثل HOG

على الصورة. هذه الخوارزمية (التي صممها إنسان) تقوم بتحليل الصورة وتنتج متجه أرقام يمثلها. مثلاً، قد يكون المتجه $[0.12, 0.85, 0.33, \dots]$ حيث يمثل كل رقم قوة ميزة معينة (مثل وجود حلقة في الأعلى، خط أفقي في الأسفل، إلخ).

■ التصنيف (آلي) :

نعطي هذا المتجه لخوارزمية تعلم آلي مثل SVM أو KNN هذه الخوارزمية تم تدريبها مسبقاً على آلاف المتجهات التي تمثل أرقاماً مختلفة (0, 1, 2, ..., 9) .

■ المخرج :

تقول خوارزمية التصنيف: "بناءً على هذا المتجه، هذه الصورة هي على الأرجح الرقم '5' " .

4 . القيد الأكبر في هذا النهج: " الميزات مصممة من قبل إنسان "

هنا تكمن النقطة الجوهرية على الرغم من أن "الآلة" هي التي تكتشف النمط النهائي، إلا أنها تعمل على بيانات (الميزات) قام الإنسان باختيارها وتصميمها.

إذاً " لا يمكن الحصول على نتائج جيدة دون استخدام ميزات مناسبة للمشكلة. "

مثال :

للتعرف على الرقم "5" : الميزات المفيدة التي قد يصممها الإنسان هي: وجود حلقة في الاسفل، خط أفقي في الاعلى. للتعرف على وجه كلب: الميزات المفيدة هنا تختلف تماماً: شكل الأذنين، ملمس الفرو، لون العينين. إذا استخدمت ميزات التعرف على الأرقام للتعرف على وجوه الكلاب، سيفشل النظام بشكل ذريع، لأنه يبحث عن خصائص غير موجودة. هذا يعني أن لكل مشكلة، يجب على الإنسان الخبير أن يجلس ويفكر ويصمم مجموعة الميزات المناسبة. هذا لا يزيل العبء عن الإنسان، بل ينقله من "كتابة الخوارزمية" إلى "تصميم الميزات".



5. المقارنة مع التعلم العميق : بدون تدخل بشري

التعلم العميق (Deep learning) عبر الشبكات العصبية

النهج السابقان:

الإنسان يكتب الخوارزمية كاملة .

الإنسان يصمم الميزات، والآلة تتعلم التصنيف.

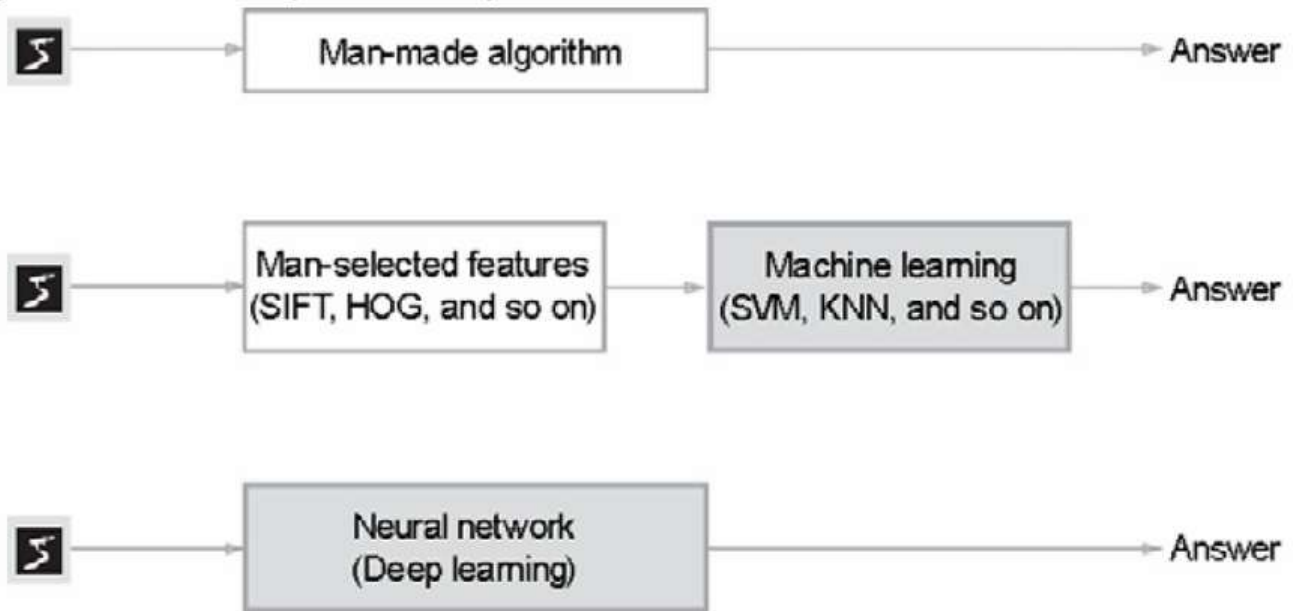
النهج الجديد :

ماذا يعني "بدون تدخل بشري" في هذا السياق؟ يعني أننا لم نعد بحاجة إلى خطوة "تصميم الميزات" التي يقوم بها الإنسان. بدلاً من إعطاء الشبكة العصبية متجه الميزات، نعطيها الصورة الخام مباشرة (مصفوفة البكسل) .

الشبكة العصبية، بفضل طبقاتها المتعددة، تتعلم بنفسها كيفية استخلاص الميزات المفيدة.

الخلاصة :

- المرحلة الأولى (العبء كله على الإنسان): كتابة الخوارزميات يدوياً.
- المرحلة الثانية (التعلم الآلي التقليدي): الإنسان يسهّل العبء عبر تصميم "مكونات" (الميزات)، والآلة تتعلم كيفية تجميعها.
- المرحلة الثالثة (التعلم العميق): الإنسان يوفر البيانات الخام والإجابات الصحيحة فقط، والآلة تتعلم كيف تستخلص المكونات وكيف تجمعها بنفسها، بدون تدخل مباشر في تصميم الميزات.



Deep learning

التعلم العميق يسمى أحياناً التعلم من طرف إلى طرف (end to end) أي من طرف إلى الطرف الآخر للحصول على النتيجة المرجوة (المخرجات) انطلاقاً من البيانات الخام (المدخلات)

- الطرف الأول (البداية): البيانات الخام (Raw Data) هي البيانات الأصلية غير المعالجة.

مثال: في التعرف على الصور، البيانات الخام هي مصفوفة قيم البكسل (الألوان) كما هي مباشرة من الكاميرا أو الماسح الضوئي. لم يتم عليها أي تحليل أو تعديل.

مثال آخر: في التعرف على الكلام، البيانات الخام هي الموجة الصوتية (Waveform) المسجلة.

- الطرف الآخر (النهاية): النتيجة المرجوة (Desired Result) هي الإجابة النهائية أو المخرجات التي نريد من النظام أن ينتجها.

مثال: في التعرف على الصور، هي كلمة "قطعة" أو الرقم "5".

مثال آخر: في ترجمة اللغات، هي الجملة المترجمة بالكامل.

إذاً، "التعلم من طرف إلى طرف" يعني قدرة النظام على أخذ البيانات في أبسط صورها الخام وتحويلها مباشرة إلى النتيجة النهائية المطلوبة، في عملية واحدة متكاملة.

مقارنة :

- النهج التقليدي (ليس من طرف إلى طرف) :

بيانات خام (صورة) - > خطوة بشرية: استخلاص الميزات [متجه أرقام] - > خطوة آلية: التصنيف - > نتيجة

لاحظ أن هناك خطوة وسيطة حاسمة يقوم بها الإنسان وهي : استخلاص الميزات . "العملية ليست متصلة من الطرف إلى الطرف"، بل تنقطع عند هذه الخطوة التي تتطلب تدخلاً بشرياً وخبرة متخصصة.

- نهج التعلم العميق (من طرف إلى طرف) :

بيانات خام (صورة) - > كتلة واحدة متكاملة: الشبكة العصبية - > نتيجة

هنا، الخطوة الوسطى اختفت. الشبكة العصبية تأخذ البيانات الخام مباشرة وتنتج النتيجة النهائية. هي نفسها، داخلياً، تقوم بما كان يقوم به الإنسان في النهج التقليدي (أي استخلاص الميزات)، ولكنها تتعلم كيفية فعل ذلك تلقائياً.

مثال:

- التعلم الآلي التقليدي:

أنت تريد أن تعلم طاهياً آلياً كيف يصنع كعكة. أنت (الخبير البشري) تقوم بعملية معقدة: تحضر المكونات الجاهزة (دقيق مُعد مسبقاً للكيك، كريمة جاهزة، فواكه مقطعة). ثم تعطي هذه المكونات الجاهزة للطاهي الآلي ليقوم فقط بتجميعها وخبزها. الطاهي الآلي لا يعرف كيف يحول الدقيق الخام إلى دقيق للكيك، بل يعتمد على عمالك المسبق.

■ التعلم من طرف إلى طرف (التعلم العميق):

أنت تعطي الطاهي الآلي (الشبكة العصبية) المكونات الخام جداً: قمح، بيض، سكر خام، حليب. المعجزة هنا هي أن هذا الطاهي الآلي لا يجمع فقط، بل يكتشف بنفسه كيف يطحن القمح، وكيف يخلط المكونات، وبأي نسب، وكيف يخبزها للحصول على كعكة مثالية. لقد تعلم العملية بأكملها من البداية إلى النهاية .

ببساطة:

التعلم من طرف الى طرف يعني القدرة على أتمتة العملية بأكملها، من أخذ البيانات الأكثر بدائية إلى إنتاج الأجوبة الأكثر تعقيداً، في تدفق مستمر ومتكامل يقلل من الحاجة إلى التدخل البشري التفصيلي .

أهم المبادئ الأساسية في التعلم الآلي، وهو مبدأ الفصل بين البيانات المستخدمة للتدريب وتلك المستخدمة للاختبار

نقسم البيانات المتاحة إلى مجموعتين حسب الغرض:

بيانات التدريب training data :

هي الجزء الأكبر من البيانات الذي نعطيه للنموذج لـ "يتعلم" منه. هي بمثابة "الكتاب الدراسي" و"مسائل التدريب" للنموذج.

الغرض منها ؟

ايجاد المعاملات المثلى خلال هذه المرحلة، يُسمح للنموذج بتحليل البيانات، ورؤية المدخلات والمخرجات الصحيحة المقابلة لها، وتعديل أوزانه الداخلية (المعاملات) مراراً وتكراراً لتقليل الخطأ قدر الإمكان. الهدف هنا هو "التدريب" بحد ذاته.

بيانات الاختبار test data :

هي مجموعة بيانات منفصلة تماماً عن بيانات التدريب. النموذج لم يرها ابداً خلال مرحلة التدريب. هي بمثابة "الامتحان النهائي" للنموذج.

الغرض منها ؟

تقييم قدرة النموذج المدرب بعد انتهاء التدريب، نستخدم هذه البيانات لنرى كيف سيتصرف النموذج مع بيانات جديدة لم يتعامل معها من قبل. هذا التقييم هو المقياس الحقيقي لجودة النموذج.

السبب الجوهرى للفصل هو (التعميم Generalization) :

لماذا نفصل بينهما ؟

"لأننا نريد القدرة على التعميم لدى النموذج".

ماهو التعميم ؟

قدرة النموذج على التعامل مع البيانات غير المعروفة (البيانات غير الموجودة في بيانات التدريب) .

لماذا التعميم هو الهدف الأسمى ؟

- لأن العالم الحقيقي مليء ببيانات جديدة وغير متوقعة
- فإذا كان النموذج جيداً فقط على البيانات التي تدرب عليها، فلا فائدة منه عملياً مهما كان أدائه ممتازاً على بيانات التدريب
- لأن الهدف من بناء أي نظام ذكاء ليس أن يتفوق في امتحان أعد له بشكل مسبق بل أن يكون مفيداً في العالم الحقيقي المليء بالبيانات غير المعروفة والمتغيرة باستمرار

مثال :

نظام يقرأ الرموز البريدية على البطاقات البريدية تلقائياً

■ المتطلب الحقيقي :

يجب أن يكون نظام التعرف على الأرقام المكتوبة بخط اليد قادراً على التعرف على الحروف المكتوبة بواسطة شخص ما

■ ما الفرق بين شخص ما وشخص محدد ؟

شخص ما : هذا يعني التعبير عن الحالة العامة يعني أن أي شخص في العالم قد يكتب هذه البطاقة : طفل يتعلم الكتابة , طبيب يكتب بسرعة , شخص يستخدم لغة أجنبية الخ هذا الشخص يمثل النوع اللانهائي للبيانات الحقيقية . النظام يجب أن يتعامل مع هذا التنوع .

شخص محدد (A specific person): هذا هو التعبير عن البيانات الموجودة في مجموعة التدريب , لنفترض أننا جمعنا 100,000 صورة مكتوبة بخط اليد من قبل 500 شخص , هؤلاء ل 500 هم محددين النموذج , قد يتعلم النموذج أنماط خطوط هؤلاء الأشخاص ال 500 بشكل ممتاز.

إذاً النموذج أصبح مثل الطالب الحافظ وهو حفظ خطوط ل 500 شخص إذا جاءت بطاقة بريدية مكتوبة من قبل الشخص 501 فقد يفشل النموذج لأنه لم يرى هذا النمط من الخط من قبل
هذه الحالة بالضبط هي ما يعرف بـ overfitting النموذج أصبح متخصصاً جداً في بيانات التدريب لدرجة أنه فقد مرونته وقدرته على التكيف مع البيانات الجديدة .

لذلك :

عندما نقوم بتقييم أي نموذج فإننا لا ننظر فقط إلى أدائه على بيانات التدريب بل الأهم من ذلك النظر إلى أدائه على بيانات الاختبار لقياس قدرته الحقيقية على التعميم

■ لمحة سريعة عن overfitting :

ماذا لو استخدمنا مجموعة بيانات واحدة فقط للتدريب والتقييم ؟

لن يتوفر التقييم الصحيح

■ لماذا ؟

لأنه من غير العدل أن نسأل النموذج أسئلة قد رأى إجاباتها من قبل سيكون أدائه مثالياً لكن هذا الأداء كاذب .

■ النتيجة :

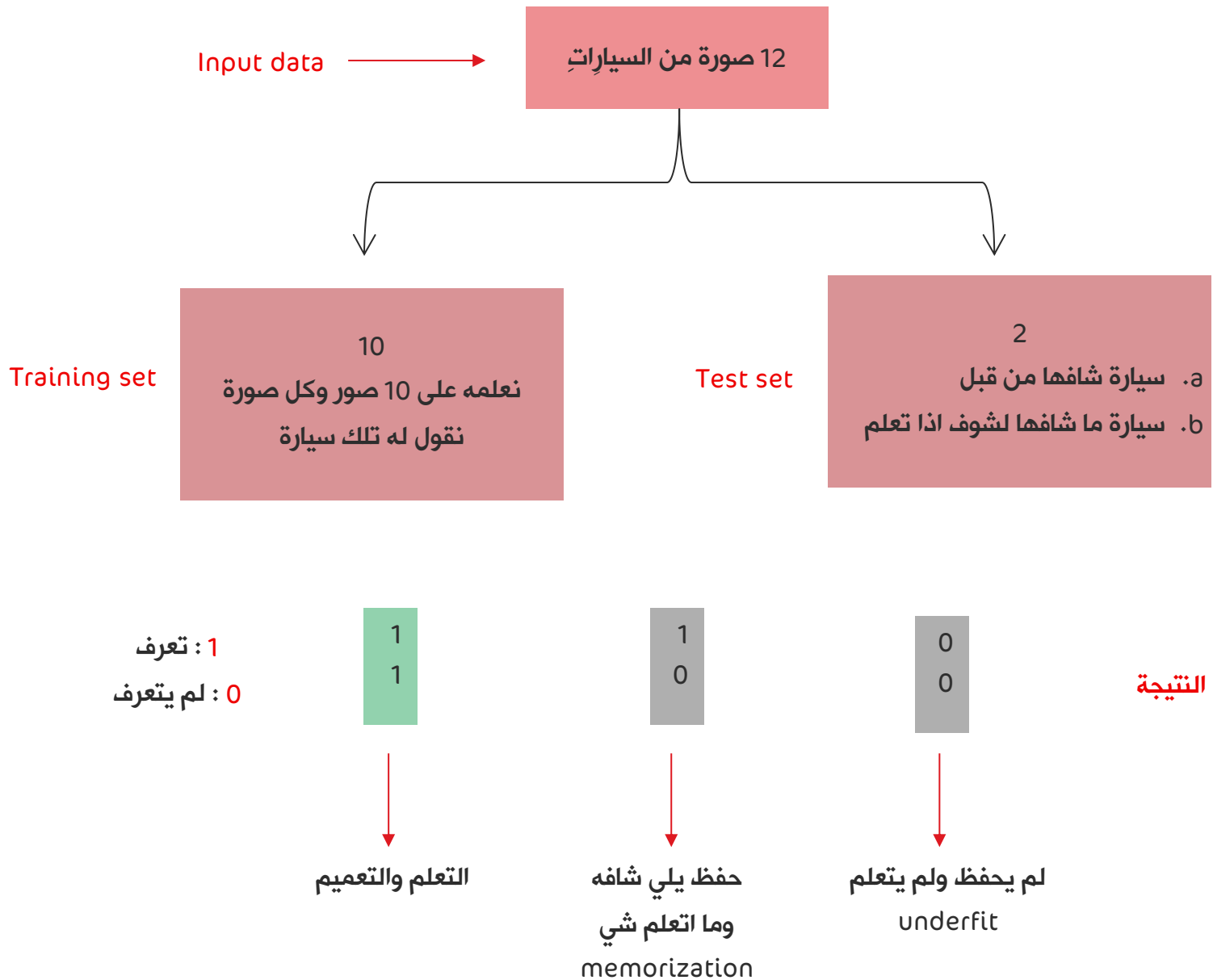
يؤدي هذا إلى نموذج يمكنه التعامل مع مجموعة بيانات معينة بشكل جيد ولكنه لا يستطيع التعامل مع مجموعة أخرى

إذاً النتائج الممتازة التي نحصل عليها هي مجرد وهم لأن النموذج حفظ بيانات التدريب بدلاً من تعلم القواعد العامة وهذا ما يعرف ب overfitting

مثال:

طالب جيد (تعميم): طالب درس المفاهيم والقواعد في كتاب الرياضيات. عندما يواجه امتحاناً يحتوي على مسائل جديدة لم يرها من قبل، يستطيع حلها لأنه فهم المبدأ.

طالب "حافظ" (overfitting): طالب لم يدرس المفاهيم، بل حفظ إجابات المسائل الموجودة في الكتاب حرفياً. إذا جاء في الامتحان نفس المسائل، سيحصل على 100%. لكن إذا تغيرت الأرقام قليلاً، سيعجز عن الحل لأنه لم يتعلم "كيف" يحل.



دالة الخسارة loss function

مثال تقريبي لفهم الفكرة :

تعلم رمي كرة السلة

تخيل أنك تتعلم رمي كرة السلة وتريد أن تصبح محترفاً.

■ الهدف (الجودة):

أن تدخل الكرة في السلة. هذا هو "الأداء الجيد".

■ المشكلة :

كيف تقيس أدائك في كل محاولة؟ هل تقول "نجحت" أو "فشلت" فقط؟ هذا لا يساعدك كثيراً. إذا فشلت، هل تعرف لماذا؟ هل يجب أن ترمي بقوة أكبر؟ أم بزاوية مختلفة؟

■ الحل (دالة الخسارة) :

لنستخدم مقياساً أفضل. بدلاً من قياس "النجاح"، سنقيس "مسافة الخطأ".

1. إذا دخلت الكرة في السلة تماماً، (مسافة الخطأ = 0). هذا هو المثالي، لا يوجد خطأ.

2. إذا سقطت الكرة على بعد 5 سم من حافة السلة، (مسافة الخطأ = 5 سم). خطأ بسيط، أداء سيء قليلاً.

3. إذا سقطت الكرة على بعد متر من السلة، (مسافة الخطأ = 100 سم). خطأ كبير، أداء سيء جداً.

هنا تكمن الفكرة "مسافة الخطأ" هذه هي دالة الخسارة الخاصة بك. إنها لا تقيس "جودة" رميتك، بل تقيس "سوء" رميتك (مدى ابتعادك عن الهدف).

■ كيف تساعدك على التحسن؟

في كل مرة ترمي الكرة، تنظر إلى "مسافة الخطأ". إذا كانت الكرة تقع دائماً على يمين السلة بمسافة 10 سم، فأنت تعرف أن مشكلتك هي الاتجاه. ستقوم بتعديل طريقة رميك قليلاً إلى اليسار في المرة القادمة لتقليل مسافة الخطأ. أنت لا تحاول "تعظيم النقاط" بشكل مباشر، بل تحاول "تقليل الخطأ". وعندما تنجح في تقليل الخطأ إلى الصفر، فإنك تلقائياً تعظم النقاط.

الشبكة العصبية تشبه تماماً هذا اللاعب

■ الشبكة العصبية : هي الطالب الذي يتعلم.

■ الأوزان (Weights) : هي "طريقة الرمي التي تستطيع الشبكة تعديلها.

■ بيانات التدريب (Training Data) : هي "محاولات الرمي" التي تتعلم منها.

■ دالة الخسارة (Loss Function) : هي "مسافة الخطأ" هي رقم واحد يخبر الشبكة: "أنت الآن بعيد جداً عن الإجابة الصحيحة" (خسارة عالية) أو "أنت قريب جداً" (خسارة منخفضة).

■ لماذا نقيس "السوء" وليس "الجودة"؟

هذه هي النقطة الذكية في الرياضيات:

قياس "الجودة" (مثلاً: دقة 95%) صعب استخدامه في العمليات الحسابية المعقدة.

قياس "السوء" (مثلاً: خطأ بنسبة 5%) سهل جداً في الرياضيات. يمكننا حساب مشتقته، ومعرفة كيفية تقليله.

إذاً تقليل الخطأ بنسبة 5% هو نفس الشيء تماماً مثل "زيادة الدقة بنسبة 95%". نحن فقط نستخدم الطريقة الأسهل للرياضيات والآلات.

إذاً، دالة الخسارة هي المدرس الخاص للشبكة العصبية. في كل مرة تجيب فيها الشبكة على سؤال، ينظر لها هذا "المدرس" ويقول لها: "إجابتك كانت بهذا المقدار من الخطأ. حاول في المرة القادمة أن تعدل أوزانك لتقلل هذا الخطأ". وهكذا، وبمرور الوقت، تتعلم الشبكة كيف تصل إلى أدنى خطأ ممكن، أي إلى أفضل أداء ممكن.

■ تعميم:

"في تدريب الشبكة العصبية، نستخدم درجة واحدة (score) للإشارة إلى الحالة الحالية. بناءً على هذه الدرجة، يتم البحث عن معاملات الوزن المثلى".

مثال: تخيل أنك تقود سيارة في ضباب كثيف ولا يمكنك رؤية الطريق. كل ما لديك هو جهاز واحد (GPS) يخبرك: "أنت على بعد 5 كيلومترات من وجهتك". هذه "الدرجة" (المسافة المتبقية) هي المقياس الوحيد الذي يوجهك. في تدريب الشبكة العصبية، الأمر مشابه. الشبكة لديها ملايين الأوزان التي يمكن تعديلها. لمعرفة ما إذا كانت التعديلات تقربها من الحل الصحيح أم تباعدتها، نستخدم رقم واحد فقط، وهو قيمة "دالة الخسارة". هذا الرقم هو الدليل الوحيد الذي يوجه عملية البحث عن الأوزان المثلى.

■ هناك طرق رياضية مختلفة لحساب هذا الرقم، لكن الأكثر شيوعاً في مجالنا هي طريقتان:

■ Sum of squared errors

■ Cross-entropy error

كلاهما يهدف إلى نفس الغاية: إعطاء رقم يمثل مقدار الخطأ. دالة الخسارة لا تقيس "مدى جودة" أداء الشبكة، بل تقيس "مدى سوءه".

"مدى عدم الملاءمة:"

إلى أي مدى تختلف مخرجات الشبكة عن الإجابات الصحيحة الموجودة في البيانات؟

"مدى الانحراف:"

كم هي المسافة بين توقع الشبكة والحقيقة؟

الربط بالمثال:

في مثال كرة السلة، كنا نقيس "مسافة الكرة من السلة". هذه المسافة هي بالضبط "سوء" الرمية. كلما كانت المسافة أكبر، كانت الرمية أسوأ. دالة الخسارة تفعل الشيء نفسه مع الشبكة العصبية. "قد تشعر أن من غير الطبيعي أن يكون 'سوء القدرة' هو الدرجة، ولكن يمكنك تفسير دالة الخسارة مضروبة في قيمة سالبة على أنها درجة عكس 'سوء القدرة' (أي درجة 'جودة القدرة')". قد يبدو غريباً أن نهتم بالسوء بدلاً من الجودة. لكن رياضياً بسيطة: إذا كان لدينا مقياس للسوء (مثلاً، قيمة الخسارة = 5)، فيمكننا بسهولة تحويله إلى مقياس للجودة بضربه في -1 (فنحصل على -5).

المعنى :

تقليل سوء القدرة هو نفس الشيء مثل 'تعظيم جودة القدرة'. لذلك، فإن مؤشر 'سوء' القدرة هو في جوهره نفس مؤشر جودة القدرة.

- عندما نقول للشبكة العصبونية: "حاول تقليل قيمة الخسارة (السوء) قدر الإمكان"، فنحن في الحقيقة نقول لها: "حاول تعظيم جودة أدائك قدر الإمكان".
- الهدف النهائي واحد: الوصول إلى أفضل أداء ممكن.
- الربط بالمثال :عندما كان لاعب كرة السلة يحاول تقليل مسافة الخطأ (السوء)، كان في الواقع يحاول تعظيم دقة رميته (الجودة). الهدف واحد، ولكننا عبرنا عنه بطريقة رياضية عملية.

الخلاصة النهائية :

دالة الخسارة هي المقياس الرسمي لـ "سوء" أداء الشبكة. نستخدم هذا المقياس لأنه من الناحية الرياضية والخوارزمية أسهل بكثير في التحسين (البحث عن الحد الأدنى) من مقياس "الجودة" (البحث عن الحد الأعلى). لكن في النهاية، تقليل الخطأ هو نفس الشيء تماماً مثل زيادة الصحة. هما وجهان لعملة واحدة، ونحن نستخدم الوجه الذي يسهل على الآلة فهمه والعمل على تحسينه .

أشهر دوال الخسارة:

Sum of squared errors SSE .1

$$E = \frac{1}{2} \sum_k (y_k - t_k)^2$$

مجموع الأخطاء المربعة هو مجموع مربعات الفروقات بين مخرجات الشبكة العصبونية والعناصر المقابلة للبيانات الصحيحة

شرح القانون :

y_k : هو المخرج الخاص بالشبكة العصبونية. ليس مجرد رقم واحد، بل هو متجه (Vector) من الأرقام. كل عنصر في هذا المتجه يمثل "ثقة" الشبكة بأن الصورة تنتمي إلى فئة معينة.

t_k : هو البيانات المصنفة (Labeled Data) أو "الإجابة الصحيحة". هو أيضاً متجه من الأرقام يمثل الحقيقة المطلقة للصورة.

K : هو عدد الأبعاد أو عدد الفئات في المشكلة. بما أننا نتعرف على الأرقام من 0 إلى 9، فهناك 10 فئات ممكنة، وبالتالي $k = 10$.

مثال :

$$y = [0.1, 0.05, 0.6, 0.0, 0.05, 0.1, 0.0, 0.1, 0.0, 0.0]$$

$$t = [0, 0, 1, 0, 0, 0, 0, 0, 0, 0]$$

شرح متجه المخرجات y :

هذا المتجه هو نتيجة دالة تسمى Softmax وظيفتها هذه الدالة هي تحويل مخرجات الشبكة الأولية إلى توزيع احتمالات . هذا يعني أن كل قيمة في المتجه y تمثل احتمالاً (Probability) ، ومجموع كل القيم في المتجه يساوي 1 (أو قريباً جداً من 1).

تفسير المثال:



$y[0] = 0.1$: الشبكة تعتقد أن هناك احتمال 10% أن تكون الصورة هي الرقم 0 .
 $y[1] = 0.05$: الشبكة تعتقد أن هناك احتمال 5% أن تكون الصورة هي الرقم 1 .
 $y[2] = 0.6$: الشبكة تعتقد أن هناك احتمال 60% أن تكون الصورة هي الرقم 2. هذه هي أعلى نسبة احتمال، وبالتالي التخمين النهائي للشبكة هو 2.
 وبالمثل لبقية الأرقام، الاحتمالات إما صفر أو منخفضة جداً.

الخلاصة :

المتجه y هو تخمين الشبكة المؤهل بالاحتمالات. هو لا يقول "الإجابة هي 2" فقط، بل يقول "أنا أثق بنسبة 60% أن الإجابة هي 2، وأثق بنسبة 5% أنها 1، وهكذا."

شرح متجه البيانات المصنفة t :

هذا المتجه هو مفتاح الإجابات أو الحقيقة المطلقة التي نستخدمها لتصحيح الشبكة.

■ الترميز (One-Hot Representation) :

هذه هي الطريقة التي يتم بها تمثيل الإجابة الصحيحة.
 نضع الرقم 1 في الـ index (الفهرس) الذي يتوافق مع الإجابة الصحيحة.
 نضع الرقم 0 في كل الفهارس الأخرى.
 تفسير المثال:
 $t = [0, 0, 1, 0, 0, 0, 0, 0, 0, 0]$
 القيمة 1 موجودة في الفهرس رقم 2 (بما أننا نبدأ العد من 0). هذا يعني أن الإجابة الصحيحة والمؤكددة 100% هي الرقم 2 .

كل القيم الأخرى هي 0، مما يعني أننا على يقين تام بأن الصورة ليست 0 ولا 1 ولا 3 وهكذا.
 هذا المثال يوضح الآلية الأساسية لتدريب الشبكات العصبونية على مشاكل التصنيف:

- الشبكة تُخرج تخميناً على شكل متجه احتمالات y .
- البيانات توفر الإجابة الصحيحة على شكل متجه ترميز t .
- دالة الخسارة تقارن بين هذين المتجهين وتُصدر رقماً واحداً يمثل مقدار الخطأ.
- عملية التدريب تهدف إلى تعديل أوزان الشبكة لتقليل هذا الخطأ، أي لجعل متجه المخرجات y يقترب أكثر فأكثر من متجه الإجابة الصحيحة t .

لنتحدث قليلاً عن One-Hot Representation :

ببساطة، هو طريقة لتحويل بيانات فئوية (Categorical Data) إلى أرقام يمكن للآلة فهمها.

■ البيانات الفئوية:

هي بيانات تصف فئة أو صنف، وليس رقماً. مثلاً: (قطعة، كلب، طائر)، أو (أحمر، أخضر، أزرق).

■ طريقة التمثيل:

- نحدد كل الفئات الممكنة.
- ننشئ متجهاً (Vector) أو قائمة من الأصفار. طول هذه القائمة يساوي عدد الفئات.
- نضع الرقم 1 في الموضع (index) الذي يتوافق مع الفئة الحالية.

▪ نترك كل المواضيع الأخرى 0 .

اسم "One-Hot" يأتي من فكرة الدوائر الكهربائية القديمة: لديك صف من المقابس (switches) ، وفي أي لحظة، مقبس واحد فقط يكون موصلاً بالتيار (ON) ، والبقية تكون مطفأة (OFF) .

لماذا نستخدم التمثيل one-Hot لماذا لا نستخدم تمثيل أبسط؟

1. لتجنب الترتيب الوهمي والعلاقات الكاذبة

لنفترض أننا استخدمنا ترميزاً بسيطاً بدلاً من one-hot

قطعة = 0

كلب = 1

طائر = 2

المشكلة : عندما نعطي هذه الأرقام لنموذج تعلم آلي، قد يفسر النموذج هذه الأرقام بشكل خاطئ ويتعلم علاقات رياضية غير موجودة في الواقع.

قد يعتقد النموذج أن:

• الطائر (2) أكبر من الكلب (1) ، والكلب (1) أكبر من القطعة (0).

• الطائر (2) هو ضعف القطعة (0)

من الناحية الرياضية هذه العلاقات لا معنى لها على الإطلاق فالطائر ليس أفضل أو أكبر من الكلب. إنها مجرد فئات مختلفة.

كيف يحل one-hot هذه المشكلة؟ باستخدام $[1,0,0]$ ، $[0,1,0]$ ، $[0,0,1]$ ، كل فئة تكون على بعد متساو من الأخرى. لا توجد أي علاقة رياضية أو ترتيبية بينها. هي مجرد حالات مختلفة. النموذج يرى أنها كلها متساوية في القيمة، لكنها مختلفة في الموضع.

2. التوافق مع دوال الخسارة

3. الوضوح والدقة للنموذج :

الترميز لا يترك أي مجال للغموض عندما يرى النموذج $[0, 1, 0]$ فإنه يعرف بشكل قاطع أن الفئة هي الكلب. لا يوجد تفسير آخر ممكن. هذا يسهل على النموذج التعلم، لأنه لا يضطر إلى فك تشفير معنى الأرقام، بل يركز فقط على ربط المدخلات (مثل صورة الحيوان) بهذا المتجه الواضح.

الخلاصة :

الترميز هو تقنية ذكية وحاسمة لتحويل البيانات الفئوية إلى شكل رقمي يفهمه الحاسوب، ويجنبها الوقوع في فخ العلاقات الوهمية، ويجعلها متوافقة مع أهم دوال الخسارة المستخدمة في تدريب الشبكات العصبية.

```

3 def sum_squared_error(y, t):
4     return 0.5 * np.sum((y-t)**2)
5
6 # Assume that "2" is correct
7 t = [0, 0, 1, 0, 0, 0, 0, 0, 0]
8 # Example 1: "2" is the most probable (0.6)
9 y = [0.1, 0.05, 0.6, 0.0, 0.05, 0.1, 0.0, 0.1, 0.0]
10 print(sum_squared_error(np.array(y), np.array(t)))
11

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

0.09750000000000003

2. (Cross-Entropy Error - CEE)

$$E = -\sum(t_k * \log(y_k))$$

y_k (مخرجات الشبكة): تحدثنا عنه

t_k (التسمية الصحيحة) ايضا تحدثنا عنه

كيف تعمل؟

إذا كانت الشبكة واثقة جداً من الإجابة الصحيحة (مثلاً، y للرقم الصحيح 2 هو 0.99)، فإن $-\log(0.99)$ تكون قيمة صغيرة جداً (خطأ قليل).

أما إذا كانت الشبكة واثقة من إجابة خاطئة أو غير متأكدة من الإجابة الصحيحة (مثلاً، y للرقم 2 هو 0.01)، فإن $-\log(0.01)$ تكون قيمة كبيرة جداً (خطأ كبير).

في t_k (نستخدم التمثيل one-hot) يكون الفهرس الخاص بالتسمية الصحيحة هو 1 فقط الفهارس الأخرى هي 0. لذلك، فإن المعادلة تحسب فقط لوغاريتم المخرج الذي يتوافق مع التسمية الصحيحة. هذه هي النقطة الأكثر أهمية في هذه الدالة لنر كيف تعمل المعادلة خطوة بخطوة: لنفترض أن الفئات هي (0, 1, 2, ...). المعادلة الكاملة هي:

$$E = -(t_0 * \log(y_0) + t_1 * \log(y_1) + t_2 * \log(y_2) + \dots)$$

لنفترض أن الإجابة الصحيحة هي الرقم "2". إذاً:

$$t = [0, 0, 1, 0, \dots]$$

عندما نعوض قيم t في المعادلة، سنجد أن كل الحدود التي تضرب في صفر ستختفي:

$$E = -(0 * \log(y_0) + 0 * \log(y_1) + 1 * \log(y_2) + 0 * \log(y_3) + \dots)$$

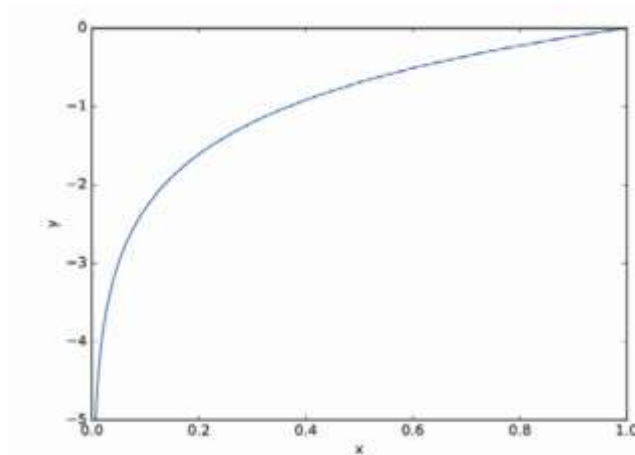
$$E = -\log(y_2)$$

الخلاصة:

المعادلة المعقدة تتبسط إلى معادلة بسيطة جداً: خطأ الانتروبي المتقاطعة هو سالب لوغاريتم احتمال الشبكة للإجابة الصحيحة.

الدالة تهتم فقط بمدى ثقة الشبكة بالإجابة الصحيحة، وتتجاهل تماماً ثقتها بالإجابات الخاطئة.

إذا تعتمد الدالة على نتيجة المخرج من التسمية الصحيحة.

**الشرح :**

عندما تكون الثقة عالية x قريب من 1 : إذا كانت الشبكة واثقة 100 % ($y_correct = 1$) ، فإن الخطأ هو $-\log(1) = 0$ هذا هو المثالي، لا يوجد خطأ على الإطلاق. كلما زادت ثقة الشبكة (اقتربت x من 1)، اقترب الخطأ من الصفر.

عندما تكون الثقة منخفضة x قريب من 0 : إذا كانت الشبكة غير واثقة أبداً ($y_correct = 0$) ، فإن الخطأ هو $-\log(0)$ رياضياً، هذا يساوي اللانهاية الموجبة. هذا يعني انه كلما قلت ثقة الشبكة بالجواب الصحيح (اقتربت x من 0)، زاد الخطأ بشكل هائل ومتصاعد .

```
def cross_entropy_error(y, t):
    delta = 1e-7
    return -np.sum(t * np.log(y + delta))
```

لماذا نحتاج الى delta ؟ المشكلة والحل : ($\delta=1e-7$):

المشكلة النظرية :

رياضياً لوغاريتم الصفر ($\log(0)$) غير معرف وقيمه هي ناقص ما لا نهاية

المشكلة العملية :

عندما تحاول مكتبة Numpy حساب $\log(0)$ فإنها لا تتعطل فحسب، بل ترجع قيمة خاصة تسمى $-\inf$ (ناقص ما لا نهاية).

إذا حاولت إجراء أي عملية حسابية لاحقة مع هذا الرقم (مثل الجمع أو الضرب)،

فستصبح جميع النتائج NaN (not a number)

وعندها يتوقف البرنامج عن العمل بشكل صحيح

الحل العملي : إضافة delta

عندما يكون أحد احتمالات الشبكة y هو صفر تماماً (مثلاً الشبكة متأكدة 100% أن الصورة ليست 7)، فإن الحساب يصبح $\log(0)$ ويفشل .

بإضافة delta، يصبح الحساب $\log(0 + 1e-7)$ أي $\log(1e-7)$ وهذا رقم سالب كبير جداً ولكنه رقم حقيقي يمكن التعامل معه.

هل هذا يغير النتيجة؟ بما أن $1e-7$ رقم صغير للغاية، فإن تأثيره على الاحتمالات الأخرى (مثل 0.6 أو 0.1) ضئيل جداً ومهملاً. إنه مجرد ترفيع تقني يمنع الكود من الانهيار دون التأثير على دقة النتيجة النهائية.

أمثلة :

```
18 t = [0, 0, 1, 0, 0, 0, 0, 0, 0, 0]
19 y = [0.1, 0.05, 0.6, 0.0, 0.05, 0.1, 0.0, 0.1, 0.0, 0.0]
20 print(cross_entropy_error(np.array(y), np.array(t)))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

0.510825457099338

الشرح :

قيمة الخطأ 0.51، وهي قيمة منخفضة، مما يعني أن "سوء" أداء الشبكة قليل، لأنها كانت واثقة من الإجابة الصحيحة

```
18 t = [0, 0, 1, 0, 0, 0, 0, 0, 0, 0]
19 y = [0.1, 0.05, 0.1, 0.0, 0.05, 0.1, 0.0, 0.6, 0.0, 0.0]
20 print(cross_entropy_error(np.array(y), np.array(t)))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

2.302584092994546

الشرح:

قيمة الخطأ 2.30، وهي قيمة مرتفعة جداً، مما يعني أن "سوء" أداء الشبكة كبير، لأنها كانت غير واثقة من الإجابة الصحيحة

مقارنة بين sum of squared and cross_entropy من حيث الاستخدام :

- SSE : مشاكل الانحدار Regression - التنبؤ بقيمة مستمرة مثل التنبؤ بأسعار المنازل
- CEE : التصنيف classification - التنبؤ بفئة مثل التعرف على الأرقام و تصنيف الصور

التدريب على دفعات صغيرة Mini_Batch Training :

المشكلة :

حساب دالة الخسارة على مجموعة بيانات ضخمة مثل MNIST (60,000 صورة) في كل مرة لتحديث الأوزان يكون بطيئاً جداً وغير عملي



الحل : التدريب على دفعات صغيرة Mini_Batch :

الفكرة :

بدلاً من استخدام كل بيانات التدريب مرة واحدة، نختار عشوائياً مجموعة صغيرة (Batch) منها (مثلاً، 100 صورة).

العملية:

1. نختار دفعة صغيرة عشوائياً من بيانات التدريب.
2. نحسب متوسط دالة الخسارة على هذه الدفعة.
3. نستخدم هذا المتوسط لتحديث معاملات الشبكة.
4. نكرر العملية.

لماذا تعمل؟ متوسط الخسارة على دفعة صغيرة عشوائية يكون تقريباً جيداً لمتوسط الخسارة على مجموعة البيانات بأكملها، ولكن الحساب أسرع بكثير.

في مشكلة تعلم الآلة (machine learning)، نستخدم جميع بيانات التدريب للتدريب. وبشكل أدق، يعني ذلك إيجاد دالة الخسارة لبيانات التدريب وإيجاد المعاملات التي تجعل هذه القيمة صغيرة قدر الإمكان. لذلك، يجب استخدام كل بيانات التدريب للحصول على دالة الخسارة.

الشرح:

لكي نضمن أننا نجد أفضل معاملات (أوزان) للشبكة، يجب أن نأخذ في الاعتبار خطأ الشبكة على كل مثال تدريب موجود لدينا.

الفكرة:

إذا كان لدينا 100 صورة، فإننا نحسب خطأ الشبكة على الصورة الأولى، ثم الثانية، ثم الثالثة... حتى المئة. ثم نجمع كل هذه الأخطاء معاً لنحصل على "الخطأ الإجمالي" للشبكة على مجموعة البيانات بأكملها.

الهدف:

نحاول تعديل الأوزان لتقليل هذا "الخطأ الإجمالي" الكبير.

في مثال دالة الخسارة الذي تحدثنا عنه سابقاً، تم استخدام دالة الخسارة لقطعة بيانات واحدة. بالنسبة لخطأ الانتروبي المتقاطعة، يمكن للمعادلة أدناه حساب مجموع دوال الخسارة لكل بيانات التدريب. في النهاية، يتم القسمة على N من أجل التطبيع (Normaliation). القسمة على N تحسب "متوسط دالة الخسارة" لكل بيانات.

المعادلة الرياضية للخطأ الإجمالي :

$$E = -\frac{1}{N} \sum_n \sum_k t_{nk} \log(y_{nk})$$

N : هو العدد الإجمالي لبيانات التدريب (مثلاً، 100 صورة).

المجموع المزدوج : المجموع الداخلي (k) يحسب خطأ الانتروبي المتقاطعة لـ صورة واحدة (كما شرحنا سابقاً). المجموع الخارجي (n) يجمع خطأ كل الصور الـ N.

(1/N) أو التطبيع : هذا هو الجزء الأهم. بدلاً من استخدام مجموع الأخطاء ، نستخدم متوسط الخطأ .

لماذا هذا مهم؟

لجعل المقياس ثابتاً بغض النظر عن حجم البيانات.
مثال: إذا كان متوسط الخطأ هو 0.5 لكل صورة، فإن:

- مع 1000 صورة، يكون المجموع 500.
 - مع 10,000 صورة، يكون المجموع 5000.
- الرقم 5000 يبدو أسوأ بكثير من 500، لكن الحقيقة أن أداء النموذج هو نفسه في كلتا الحالتين (متوسط الخطأ 0.5 لكل صورة). باستخدام المتوسط، نحصل على مقياس موثوق لا يتغير مع حجم البيانات.

المشكلة العملية : عدم جدوى استخدام كل البيانات

تحتوي مجموعة بيانات MNIST على 60,000 عنصر من بيانات التدريب. إن حساب مجموع دوال الخسارة لكل هذه البيانات يستغرق بعض الوقت. في بعض الأحيان، تحتوي البيانات الضخمة على ملايين أو عشرات الملايين من قطع البيانات. في هذه الحالة، لا يكون حساب دوال الخسارة لكل البيانات عملياً.

الشرح:

هنا تكمن المشكلة الكبرى.

- مثال: MNIST لكي نقوم بتحديث واحد فقط لأوزان الشبكة، يجب أن نمرر كل الـ 60,000 صورة عبر الشبكة، ونحسب خطأ كل واحدة، ثم نجمعها. هذه العملية ستكون بطيئة جداً
- مشكلة "البيانات الضخمة: (Big Data) "في التطبيقات الحقيقية، قد يكون لدينا ملايين الصور، حساب الخطأ على كل هذه الملايين لكل تحديث واحد سيستغرق أياماً أو أسابيع، مما يجعل التدريب مستحيلًا عملياً.

الحل - نهج الدفعات الصغيرة (Mini-Batch)

- يتم استخلاص بعض البيانات لتقريب كل البيانات. أيضاً، في تدريب الشبكات العصبية، يتم اختيار بعض بيانات التدريب، ويتم إجراء التدريب لكل مجموعة من البيانات، والتي تسمى دفعة صغيرة (mini-batch). على سبيل المثال، يتم اختيار 100 قطعة من البيانات بشكل عشوائي من 60,000 عنصر من بيانات التدريب لاستخدامها في التدريب. يسمى أسلوب التدريب هذا بالتدريب على دفعات صغيرة.
- لماذا تقريب (Approximation) ؟ الخطأ المحسوب من 100 صورة ليس هو الخطأ الحقيقي لكل الـ 60,000 صورة، ولكنه تقريب جيد جداً له. على المدى الطويل، متوسط كل هذه التقديرات العشوائية سيقترب من القيمة الحقيقية.
- لماذا عشوائي ؟ الاختيار العشوائي يضمن أن كل دفعة صغيرة تكون ممثلة لمجموعة البيانات بأكملها، مما يمنع الشبكة من أن تتعلم نمطاً متحيزاً من مجموعة فرعية معينة

مثال:

الطالب والامتحان النهائي

تخيل أنك طالب، وعندك كتاب ضخم جداً (1000 صفحة) لدراسة مادة الامتحان النهائي. هدفك هو أن تفهم المادة جيداً لتنجح في الامتحان. الدراسة هنا هي عملية تحديث معلوماتك، تماماً مثل التدريب هو عملية تحديث أوزان الشبكة العصبية.

لديك ثلاث طرق لدراسة هذا الكتاب:

■ الطريقة الأولى:

قراءة الكتاب كاملاً في كل مرة (الطريقة مثالية ولكن مستحيلة)

الفكرة: كل يوم، قبل أن تذاكر، تقرأ الكتاب كله من البداية إلى النهاية.

الميزة: سيكون فهمك للمادة مترابطاً وعميقاً جداً. سترى العلاقات بين كل الفصول.

المشكلة: هذه الطريقة تستغرق وقتاً وطاقة هائلين. بحلول وقت الامتحان، ربما تكون قد أنهيت القراءة مرة واحدة فقط. إنها غير عملية على الإطلاق.

هذه الطريقة تشبه: حساب خطأ الشبكة على كل بيانات التدريب (60,000 صورة) في كل مرة نريد تحديث الأوزان. الأمر دقيق للغاية ولكنه بطيء جداً لدرجة الاستحالة.

■ الطريقة الثانية:

قراءة جملة عشوائية كل يوم (الطريقة سريعة ولكن سيئة)

الفكرة: كل يوم، تفتح الكتاب في صفحة عشوائية وتقرأ جملة واحدة فقط.

الميزة: سريعة جداً ولا تتطلب أي مجهود.

المشكلة: معلوماتك ستكون مشتتة وغير مترابطة. قد تقرأ يوماً عن الجاذبية، وفي اليوم التالي عن التمثيل الضوئي. لن تتكون لديك صورة كاملة عن المادة. معلوماتك ستكون فوضوية وغير مستقرة.

هذه الطريقة تشبه: تحديث أوزان الشبكة بناءً على مثال تدريب واحد فقط في كل مرة. التحديث سريع، لكن "اتجاه" التعلم سيكون متقلباً جداً وغير موثوق. مثال واحد غريب قد يضلل الشبكة ويجعلها تتعلم الشيء الخطأ.

■ الطريقة الثالثة:

قراءة فصل كامل كل يوم (الطريقة ذكية وعملية)

الفكرة: بدلاً من قراءة الكتاب كله أو جملة واحدة، تقرر أن تدرس فصلاً كاملاً (حوالي 20-30 صفحة) كل يوم. وتختار هذا الفصل بشكل عشوائي من الكتاب.

لماذا هي الطريقة المثلى؟

عينة جيدة: قراءة فصل كامل تعطيك صورة متكاملة عن موضوع معين. هذه العينة (الفصل) هي تمثيل جيد لمحتوى الكتاب كله.

سريعة وفعالة: أسرع بكثير من قراءة الكتاب كله. في غضون شهر، يمكنك تغطية الكتاب بأكمله عدة مرات.

متوازنة: لأنك تختار الفصول عشوائياً، تضمن أنك تركز فقط على الفصول السهلة في البداية، بل تحصل على فهم متوازن للمادة بأكملها.

هذه الطريقة هي بالضبط "التدريب على دفعات صغيرة (Mini-Batch Training) " :

■ الكتاب الكامل هو مجموعة بيانات التدريب الكاملة (60,000 صورة) .

■ الفصل الواحد هو الدفعة الصغيرة (Mini-Batch) مثلاً، (100 صورة) .

- عملية "دراسة الفصل" هي حساب متوسط الخطأ على هذه الـ 100 صورة واستخدامه لتحديث أوزان الشبكة مرة واحدة.
- اختيار الفصول عشوائياً هو اختيار دفعة صغيرة عشوائية من بيانات التدريب في كل مرة.

مثال :

```

28 X, y = fetch_openml('mnist_784', return_X_y=True, as_frame = False)
29 x_train, x_test, t_train, t_test = train_test_split(X, y, test_size=10000, shuffle=False)
30 # Convert to integers
31 t_train = t_train.astype(int)
32 t_test = t_test.astype(int)
33 # Create encoder
34 encoder = OneHotEncoder(sparse_output=False)
35 # Fit and transform
36 t_train = encoder.fit_transform(t_train.reshape(-1, 1))
37 t_test = encoder.transform(t_test.reshape(-1, 1))

```

الشرح :

هذا المثال تحدثنا عنه في المحاضرة السابقة بالتفصيل لكن يوجد اضافات لنشرح الاضافات :

[`encoder = OneHotEncoder\(sparse\_output=False\)`](#)

قمنا بإنشاء اوبجكت من كلاس OneHotEncoder

[`sparse\_output=False`](#):

بشكل افتراضي يعطي scikit-learn مخرجات على شكل مصفوفة متفرقة (sparse matrix) لتوفير الذاكرة (لأنها مليئة بأصفار) لكننا نطلب منه أن يعطينا مصفوفة NumPy عادية (dense array) لأنها أسهل في التعامل

[`t\_train = encoder.fit\_transform\(t\_train.reshape\(-1, 1\)\)`](#)

OneHotEncoder يتوقع أن تكون البيانات على شكل مصفوفة من عمود واحد

ومتجه t_train الحالي هو (60000,) (بعد واحد)

ف reshape تحوِّله إلى (60000, 1) (60000 صف وعمود واحد)

مثال [5, 0, 4] : تصبح [[5], [0], [4]]

1. `fit_transform` : تقوم بشيئين

fit : التعلم :

يفحص الـ Encoder بيانات t_train ويتعرّف على جميع الفئات الموجودة (اي سيكتشف الأرقام من 0 إلى 9) ويقوم بإنشاء خريطة داخلية لكل فئة (الخريطة تربط الرقم الأصلي ب المتجه الجديد (الذي يحتوي على 0 و 1)) فعندما نستدعي `encoder.fit(...)` أنت تقول للـ Encoder اذهب وافحص كل الأرقام الموجودة في بيانات التدريب t_train واصنع لي قائمة بكل الفئات الموجودة، ثم أنشئ خريطة لكل فئة

transform : التحويل :

بعد أن تعلم، يطبق التحويل (One-Hot Encoding) على البيانات

النتيجة t_train: الآن لم يعد متجهاً من الأرقام، بل أصبح مصفوفة بحجم (10, 60000) كل صف فيها هو متجه يحتوي على 9 أصفار ورقم واحد

`t_test = encoder.transform(t_test.reshape(-1, 1))`

لاحظ أننا هنا نستخدم `transform` فقط وليس `fit_transform`

لماذا؟

لأن الـ Encoder تدرب وتعلم بالفعل من بيانات التدريب هو الآن يعرف أن الفئات هي 0, 1, 2, ... , 9 ومهمته هي فقط تطبيق نفس التحويل الذي تعلمه على بيانات الاختبار لو استخدمنا `fit` على بيانات الاختبار، فهذا يعتبر تسريب للمعلومات (data leakage) لأننا نسمح لنموذجنا برؤية معلومات من مجموعة الاختبار (التي يجب أن تكون مجهولة) أثناء مرحلة الإعداد هذا يؤدي إلى نتائج غير واقعية ومبالغ فيها

تطبيع الميزات (normalize features value)

```
39 #normalize features value
40 x_train = x_train.astype(np.float32)
41 x_test = x_test.astype(np.float32)
42 x_train /= 255.0
43 x_test /= 255.0
```

هنا يتم تحويل قيم وحدات البكسل (Pixels) في الصور من المدى [0, 255] إلى المدى [0.0, 1.0] هذا التطبيع مهم جداً لتسريع وتحسين استقرار عملية تدريب الشبكات العصبية.

```
x_train = x_train.astype(np.float32)
x_test = x_test.astype(np.float32)
```

تحويل بيانات الصور (`x_train`, `x_test`) إلى (Float) بحجم 32 بت، وهو مطلوب لعملية القسمة والتطبيع.

```
x_train /= 255.0    x_test /= 255.0
```

كل قيمة بكسل في الصور كانت أصلاً عدداً صحيحاً بين 0 و 255. بقسمتها على 255.0، نحولها إلى رقم عشري بين 0.0 و 1.0.

- مثال: بكسل بقيمة 255 سيصبح 1.0.
- مثال: بكسل بقيمة 127 سيصبح حوالي 0.498.

كيفية استخراج عدد محدد من عينات البيانات عشوائياً من مجموعة التدريب الكلية. هذه العينات المحددة هي الـ Mini-Batch :

```
69 train_size = x_train.shape[0]
70 batch_size = 10
71 batch_mask = np.random.choice(train_size, batch_size)
72 x_batch = x_train[batch_mask]
73 t_batch = t_train[batch_mask]
```

▪ : train_size = x_train.shape[0]

x_train هي مصفوفة تحتوي على كل صور التدريب (60,000 صورة).
shape[0] يعطينا عدد الصفوف في هذه المصفوفة، وهو بالضبط عدد الصور train_size . يساوي 60,000.

أي :

x_train : هو جدول الصور (60,000 صورة).

x_train.shape : يسأل الجدول عن أبعاده فيجيب (60000, 784) .

x_train.shape[0] : يأخذ الرقم الأول من الإجابة، وهو 60000 .

إذًا، السطر train_size = x_train.shape[0] يعني ببساطة: اجعل المتغير train_size يساوي عدد الصور الإجمالي في مجموعة التدريب، وهو 60,000 .
هذا أفضل من كتابة الرقم 60000 يدوياً، لأن الكود سيعمل بشكل صحيح حتى لو تغير حجم مجموعة البيانات في المستقبل.

▪ : batch_size = 10

هنا نحدد حجم الدفعة الصغيرة التي نريد اختيارها في هذه المرة. اخترنا 10 صور.

▪ batch_mask = np.random.choice(train_size, batch_size)

هذا أهم سطر

لنشرح مثال لفهم هذا السطر :

تخيل أن مجموعة بيانات التدريب x_train هي مكتبة ضخمة جداً تحتوي على 60,000 كتاب (كل كتاب يمثل صورة).
كل كتاب له رقم فريد أو عنوان على الرفوف (Index) . الأرقام من 0 إلى 59,999 .
train_size : هو متغير يحمل العدد الإجمالي للكتب في المكتبة، وهو 60,000 .
batch_size : هو متغير يحدد كم كتاب تريد أن تستعيروها في هذه المرة لقراءتها. لنفترض أنه 10 كتب.
الآن، لنفهم السطر batch_mask = np.random.choice(train_size, batch_size)
هذا السطر هو كأنك تطلب من أمين مكتبة ذكي جداً أن يقوم بمهمة محددة.

الأمر الذي تعطينه لأمين المكتبة هو: "من بين كل الكتب الـ 60,000 الموجودة في المكتبة، اختر لي 10 كتب بشكل عشوائي".

الأداة (np.random.choice) : هي هذا الأمين الذكي. وظيفته هي القيام بالسحب العشوائي.

المعاملات (المدخلات) :

train_size (60,000) : هو مجال البحث. أنت تخبر الأمين: "ابحث في هذه الكمية من الكتب".
batch_size (10) : هو "عدد الاختيارات". أنت تخبر الأمين: "اختر لي هذا العدد من الكتب".

النتيجة (batch_mask) :

هنا تكمن النقطة الأهم التي يجب أن تفهمها.
ما يعطيه لك الأمين؟ هل يعطيك الكتب العشرة نفسها؟ لا.
النتيجة (batch_mask) : هو قائمة بأرقام الرفوف أو أرقام الكتب التي اختارها. هذه القائمة تسمى "قناع (mask) لأننا سنستخدمها في الخطوة التالية لنخفي كل الكتب ونظهر فقط الكتب التي حددها هذا القناع.

مثال عملي :

لنفترض أن الأمر تم تنفيذه.

- المدخلات np.random.choice : (60000, 10)
- الناتج الممكن (batch_mask) : قد تكون قائمة عشوائية مثل :
[12345, 8, 50123, 59999, 400, 29010, 8888, 1, 45000, 33333]

تفسير الناتج:

- هذه الأرقام ليست هي الصور، ولا هي قيم البكسلات.
- هذه الأرقام هي فهارس (Indexes) أو عناوين 10 صور عشوائية من بين الـ 60,000 صورة الموجودة في مجموعة التدريب.
- المعنى: اذهب إلى الصورة رقم 12345، والصورة رقم 8، والصورة رقم 50123، وهكذا.

$x_batch = x_train[batch_mask]$ و $t_batch = t_train[batch_mask]$
هما التنفيذ الفعلي لعملية اختيار الدفعة الصغيرة. بعد أن جهزنا "قائمة الأرقام العشوائية (batch_mask)" ، نستخدمها الآن كمفتاح لسحب البيانات الفعلية من المكتبتين الكبيرتين.

تذكر دائماً :

- x_train : هي مكتبة الصور الكبيرة (60,000 صورة).
- t_train : هي مكتبة البطاقات الفهرسة (Catalog) المقابلة لها (60,000 بطاقة).
- batch_mask : هي قائمة الأرقام العشوائية التي اخترتها في الخطوة السابقة [12345, 8, 50123, ...].

السطر الأول :

$x_batch = x_train[batch_mask]$
هذا السطر هو كأنك تعطي أمين المكتبة قائمة الأرقام وتطلب منه الكتب.

الشرح:

أنت لا تعطيه قائمة الأرقام كبيانات، بل تعطيها كأمر فهرسة.
 الأمر الذي تعطيه لمكتبة الصور (x_train) : من بين كل الرفوف في المكتبة، اذهب إلى الرفوف ذات الأرقام التالية : 12345 ، ثم 8، ثم 50123 ... وأخرج لي الكتب الموجودة في هذه الأماكن.
 النتيجة (x_batch) : تكون صندوق صغير يحتوي على الصور العشرة التي طلبتها بالضبط. حجمه يكون (10, 784)
 (10 صور، كل صورة 784 بكسل)

هذه العملية تسمى الفهرسة المتقدمة (Advanced Indexing) أنت تستخدم مصفوفة ($batch_mask$) لفهرسة مصفوفة أخرى (x_train) ، والنتيجة هي مصفوفة جديدة تحتوي على الصفوف المحددة.

السطر الثاني:

$t_batch = t_train[batch_mask]$

هذا السطر هو كأنك تطلب من نفس الأمين البطاقات الفهرسة لنفس الكتب التي أخذها للتو.

الشرح:

أنت تعطي أمين المكتبة نفس قائمة الأرقام بالضبط.
 الأمر الذي تعطيه لمكتبة البطاقات (t_train) : الآن، بالنسبة لكل كتاب من الكتب العشرة التي سحبتها للتو، اذهب إلى ملف البطاقات الفهرسة وأخرج لي البطاقة المقابلة له .
 النتيجة (t_batch) : تكون صندوق صغير آخر يحتوي على البطاقات الفهرسة العشرة التي تتوافق تماماً مع الصور العشرة في x_batch حجمها يكون (10, 10)
 (10 تسميات، كل تسمية هي متجه one_hot بطول 10).

الخلاصة العملية :

- أولاً : نسأل عن العدد الإجمالي للكتب ($train_size = x_train.shape[0]$) .
 - ثانياً : نختار عشوائياً أرقام أرفف معينة ($batch_mask = np.random.choice(...)$) .
 - ثالثاً : نذهب إلى مكتبة الصور ونأخذ الكتب الموجودة في هذه الأرفف
 $(x_batch = x_train[batch_mask])$
 - رابعاً : نذهب إلى مكتبة التسميات ونأخذ البطاقات المقابلة لنفس الكتب
 $(t_batch = t_train[batch_mask])$
- الآن لديك x_batch و t_batch ، وهما مجموعتان صغيرتان ومتناظرتان جاهزتان لعملية التدريب على هذه الدفعة الصغيرة.

تطبيق دالة خسارة مرنة للدفعات (Implementing Cross-Entropy Error for Batches):

1. عندما تكون التسميات بصيغة التمثيل (One-Hot):

الشرح:

```
52 def cross_entropy_error(y, t):
53     if y.ndim == 1:
54         t = t.reshape(1, t.size)
55         y = y.reshape(1, y.size)
56         batch_size = y.shape[0]
57         return -np.sum(t * np.log(y + 1e-7)) / batch_size
```

1. المشكلة التي تحلها الدالة:

يجب أن تعمل الدالة بشكل صحيح سواء أعطيتها بيانات واحدة (دفعة بحجم 1) أو دفعة كبيرة (بحجم 100).

2. if y.ndim == 1

y.ndim تعطي عدد أبعاد المصفوفة y إذا كانت y تمثل بيانات واحدة، فإنها تكون مصفوفة أحادية البعد (1D) أما إذا كانت تمثل دفعة، فثنائية البعد (2D).

3. الحيلة الذكية (reshape)

- إذا كانت المدخلات بيانات واحدة: فإن y تتحول إلى مصفوفة من شكل (1,10) (صف واحد، 10 أعمدة).
- إذا كانت المدخلات دفعة بحجم 100: فإن y تكون مصفوفة من شكل (100,10). (100 صف، 10 أعمدة).

4. batch_size = y.shape[0]

كما شرحنا سابقاً، shape يسأل مصفوفة عن أبعادها ويعطيك إجابة على شكل مصفوفة (tuple).
 لحالة بيانات واحدة y.shape: سيعطي (1, 10).
 لحالة دفعة بحجم 100 y.shape: سيعطي (100, 10).

ماذا يفعل [0] y.shape؟

[0]. يأخذ العنصر الأول من مصفوفة الأبعاد.

- لحالة بيانات واحدة y.shape[0]: يأخذ الرقم الأول من (1, 10). الناتج هو 1.
- لحالة دفعة بحجم 100 y.shape[0]: يأخذ الرقم الأول من (100, 10). الناتج هو 100.

الخلاصة والغاية من السطر:

السطر batch_size = y.shape[0] يعني ببساطة:

اجعل المتغير batch_size يساوي عدد الصفوف في مصفوفة المخرجات y.

وعدد الصفوف في مصفوفة المخرجات هو بالتأكيد نفسه عدد عينات البيانات (أو الصور) في الدفعة الحالية.

إذا:

بدلاً من كتابة دالة منفصلة لحجم دفعة ثابت (مثلاً، 100)، هذه الدالة تعمل مع أي حجم

return -np.sum(t * np.log(y + 1e-7)) / batch_size

هذا هو نفس حساب خطأ الانتروبي المتقاطعة الذي شرحناه سابقاً، ولكنه الآن يعمل على المصفوفة بأكملها دفعة واحدة.

: / batch_size

هذا هو التطبيع (Normalization). نحن نقسم على حجم الدفعة لنحصل على متوسط الخطأ لكل صورة. هذا يجعل مقياس الخسارة متساوياً بغض النظر عن حجم الدفعة.

2. التعامل مع التسميات الرقمية مثل [2, 7, 0]

```
60 def cross_entropy_error(y, t):
61     if y.ndim == 1:
62         t = t.reshape(1, t.size)
63         y = y.reshape(1, y.size)
64         batch_size = y.shape[0]
65         return -np.sum(np.log(y[np.arange(batch_size), t] + 1e-7)) / batch_size
```

هذه هي النسخة الأكثر مرونة من الدالة، والتي لا تتطلب أن تكون التسميات بالصيغة one_hot

الشرح:

الجزء الجديد هنا هو `y[np.arange(batch_size), t]`.

هذا هو أسلوب فهرسة متقدم يسمح لك باختيار عناصر محددة من مصفوفة ثنائية الأبعاد بكفاءة هائلة.

`np.arange(batch_size)`:

- هذا يولد مصفوفة من الأرقام التسلسلية من 0 إلى `batch_size - 1`.

- مثال: إذا كان `batch_size = 5`,

فإن `np.arange(5)` تنشئ `[0, 1, 2, 3, 4]` هذه الأرقام تمثل فهارس الصفوف

: t

- هذا هو متجه التسميات الصحيحة للدفعة.

- مثال `t = [2, 7, 0, 9, 4]`:

هذه الأرقام تمثل فهارس الأعمدة للاحتتمالات الصحيحة.

`y[np.arange(batch_size), t]`

هذا هو الأمر الذي يجمع كل شيء. أنت تقول من مصفوفة المخرجات `y`، أعطني مصفوفة جديدة تحتوي على:

- القيمة من الصف 0 عند العمود 2

- القيمة من الصف 1 عند العمود 7

- القيمة من الصف 2 عند العمود 0

...وهكذا لكل الصور في الدفعة.

سينفذ هذا الأمر ويعيد مصفوفة جديدة هذه المصفوفة الجديدة تحتوي فقط على احتمالات الشبكة العصبية للإجابات

الصحيحة لكل صورة في الدفعة

الخطوة الأخيرة `np.log(...)` و `np.sum(...)` و `/batch_size`:

الآن وبعد أن استخلصنا احتمالات الإجابات الصحيحة فقط، نطبق عليهم نفس خطوات حساب خطأ الانتروبيا المتقاطعة

(اللوغاريتم، الجمع، والتطبيع).

بدلاً من كتابة حلقات for بطيئة وغير فعالة لاستخراج الاحتمالات الصحيحة، يمكننا استخدام أساليب الفهرسة المتقدمة لأداء المهمة في سطر واحد. هذا يجعل حساب دالة الخسارة للدفعات الكبيرة سريعاً وكفوئاً للغاية، وهو أمر حيوي لتدريب الشبكات العصبونية بفعالية .

لماذا نمت كتابة دالة الخسارة بطريقتين مختلفتين:

الجزء الأول: منطق الطريقة الأولى للتسميات (One-Hot) :

- الشرح: هذه هي الخدعة الرياضية التي تجعل الطريقة الأولى فعالة.
- تذكر أن مصفوفة التسميات t في التمثيل تحتوي على أصفار كثيرة وواحد واحد فقط.
- عندما نقوم بالضرب $np.log(y) * t$ ، فإن كل عملية ضرب تتضمن عنصراً من t وعنصراً من $\log(y)$.
- الخدعة: إذا كان العنصر من t هو 0، فالنتيجة ستكون 0 لأن أي رقم مضروب في 0 يساوي 0.
- النتيجة: كل قيم اللوغاريتمات الخاصة بالإجابات الخاطئة تتحول إلى أصفار وتلغى تلقائياً من عملية الجمع النهائية.
- ما يبقى؟ يبقى فقط قيمة اللوغاريتم الخاصة بالإجابة الصحيحة (التي كان مقابلها 1 في t).
- باختصار: الطريقة $np.log(y) * t$ هي طريقة ذكية وفعالة للفلتر (Filtering). هي تستخدم خاصية الضرب في الصفر "للتجاهل" كل حسابات الخطأ للفئات الخاطئة، والتركيز فقط على قيمة الخطأ للفئة الصحيحة.

الجزء الثاني: منطق الطريقة الثانية (للتسميات الرقمية)

- الشرح: لماذا نحتاج إلى طريقة ثانية؟
- المشكلة: أحياناً، لا تكون التسميات t بالصيغة $[0,0,1,0]$ ، بل تكون مجرد أرقام بسيطة [2] في هذه الحالة، لا يمكن استخدام طريقة الضرب $np.log(y) * t$ مباشرة.
- الحل: نحتاج إلى طريقة أخرى لاستخلاص احتمالات الإجابات الصحيحة من مصفوفة المخرجات y هذه الطريقة هي $y[np.arange(batch_size), t]$

- $np.arange(batch_size)$: هذا يولد مصفوفة تمثل أرقام الصفوف في الدفعة.

مثال : إذا كان $batch_size = 5$

فإن $np.arange(5)$ يولد $[0, 1, 2, 3, 4]$ هذه هي فهارس الصور من 0 إلى 4.

- t : هذه هي مصفوفة التسميات الصحيحة

مثال $t = [2, 7, 0, 9, 4]$: هذه هي فهارس الأعمدة للإجابات الصحيحة.

- $y[np.arange(batch_size), t]$:

هذا الامر يقول : اذهب إلى مصفوفة المخرجات y ، وخذ لي قائمة من الإحداثيات. كل إحداثي يتكون من (رقم صف، رقم عمود) .

الإحداثي الأول: صف 0, عمود 2.

الإحداثي الثاني: صف 1, عمود 7

الإحداثي الثالث: صف 2, عمود 0

وهكذا...

■ النتيجة : يذهب إلى هذه الإحداثيات بالضبط ويجمع لك القيم في مصفوفة جديدة.

$y[0, 2]$: قيمة الاحتمال التي أعطتها الشبكة للصورة الأولى عند الفئة 2.

$y[1, 7]$: قيمة الاحتمال التي أعطتها الشبكة للصورة الثانية عند الفئة 7.

$y[2, 0]$: قيمة الاحتمال التي أعطتها الشبكة للصورة الثالثة عند الفئة 0.

وهكذا...

■ الخلاصة : الأمر `y[np.arange(batch_size), t]`

هو طريقة سريعة وفعالة للغاية لاستخلاص فقط احتمالات الإجابات الصحيحة لكل صورة في الدفعة، وذلك في خطوة واحدة ومُحسَّنة بالكامل، دون الحاجة إلى حلقات `for` بطيئة.

لماذا نستخدم دالة الخسارة (Loss Function) بدلاً من دقة التعرف (Accuracy) لتدريب الشبكة العصبية؟

الشرح:

تخيل أنك تريد تدريب شبكة عصبية للتعرف على صور الأرقام (من 0 إلى 9). هدفك النهائي هو أن تكون الشبكة دقيقة قدر الإمكان، أي أن تعطي الإجابة الصحيحة في أغلب الأحيان.

1. المشكلة في استخدام "الدقة" مباشرة:

لماذا لا نقول للشبكة: "حسن من معاملاتك (الأوزان والانحيازات) لزيادة الدقة"؟

لأن "الدقة" مقياس غير سلس (not smooth) و غير متدرج (not continuous).

مثال: تخيل أن شبكتك تعطي 9 إجابات صحيحة من أصل 10 صور. دقتها هي 90%. الآن، إذا قمت بتعديل طفيف جداً في أحد الأوزان، فمن المرجح أن تظل الشبكة تعطي نفس 9 إجابات صحيحة. الدقة لا تزال 90%. لم تحصل على أي معلومة تخبرك هل كان تعديلك جيداً أم سيئاً.

بعبارة أخرى: الدقة تخبرك فقط إذا كنت في القمة أم لا، لكنها لا تخبرك بأي اتجاه يجب أن تخطو خطواتك التالية لتصل إلى القمة. إنها مثل إشارة "نعم/لا" فقط.

2. الحل باستخدام دالة الخسارة :

بدلاً من ذلك، نستخدم "دالة الخسارة". هذه الدالة هي مقياس رقمي وسلس يخبرنا بمدى "سوء" أداء الشبكة.

كلما كانت إجابة الشبكة خاطئة، كانت قيمة الخسارة عالية.

كلما كانت إجابة الشبكة قريبة من الصواب، كانت قيمة الخسارة منخفضة.

هدفنا لم يعد زيادة الدقة مباشرة، بل تقليل الخسارة إلى أقل ما يمكن.

3. دور المشتق (The Magic Part) :

هنا يأتي دور المشتق أو التدرج Gradient المشتق هو الذي يخبرنا الاتجاه الذي يجب أن نسلكه لتقليل الخسارة. لنستخدم تشبيه المتسلق في الجبل:

الجبل: يمثل جميع الحلول الممكنة للشبكة العصبية.

موقعك على الجبل (خط الطول والعرض): يمثل قيم الأوزان والانحيازات الحالية في الشبكة.

ارتفاعك عن سطح البحر: يمثل قيمة دالة الخسارة. هدفك هو الوصول إلى أدنى نقطة في الوادي (أقل خسارة ممكنة).

المشتقة: هو الذي يخبرك عن انحدار الأرض تحت أقدامك.

كيف يعمل المتسلق (الشبكة العصبية)؟

1. ينظر إلى انحدار الأرض تحت قدميه (يحسب المشتق).
2. يتخذ خطوة صغيرة في الاتجاه الذي ينحدر فيه الجبل أكثر (الاتجاه المعاكس للمشتق).

لنصل حالات المشتق :

إذا كان المشتق سالب (Derivative is negative) :

- المعنى: هذا يعني أنه يمكنك تقليل دالة الخسارة عن طريق تغيير معامل الوزن في الاتجاه الموجب.
 - مثال: الأرض تنحدر إلى يمينك. لتذهب إلى الأسفل، يجب أن تخطو خطوة إلى اليمين.
- إذا كان المشتق موجب (Derivative is positive) :

- الترجمة الرياضية: هذا يعني أنه يمكنك تقليل دالة الخسارة عن طريق تغيير معامل الوزن في الاتجاه السالب.

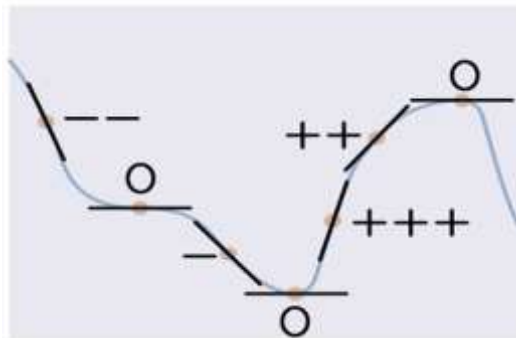
- التشبيه: الأرض تنحدر إلى يسارك. لتذهب إلى الأسفل، يجب أن تخطو خطوة إلى اليسار.

إذا كانت المشتق يساوي صفراً (Derivative is zero) :

- الترجمة الرياضية: أنت وصلت إلى نقطة حيث أي تغيير بسيط في الوزن لن يغير قيمة الخسارة.
 - التشبيه: الأرض تحت قدميك مستوية تماماً. لقد وصلت على الأرجح إلى قاع الوادي (أو نقطة صغرى محلية).
- هنا نتوقف عن التحديث.

خلاصة :

نحن لا نستخدم الدقة مباشرة لأنها لا تعطينا إرشادات دقيقة لتحسين الشبكة. بدلاً من ذلك، نستخدم دالة الخسارة كمرشد رقمي وسلس، ونستخدم المشتق كبوصلة تخبرنا بالاتجاه الصحيح في كل خطوة صغيرة لتحديث معلمات الشبكة، بهدف الوصول إلى أدنى نقطة ممكنة للخسارة، والتي تترجم في النهاية إلى أفضل أداء وأعلى دقة.



لا يمكننا استخدام دقة التعرف كمقياس لأن المشتق يصبح صفراً في معظم المواقع، مما يمنع تحديث المعاملات. الآن، دعونا نلخص هذا بشكل مرتب

المشكلة: استخدام دقة التعرف كمقياس

1. ما هي دقة التعرف؟

هي مقياس بسيط يقول: كم عدد المرات التي أجابت فيها الشبكة بشكل صحيح؟. على سبيل المثال، إذا كانت الشبكة قد تعرفت على 32 صورة بشكل صحيح من أصل 100 صورة، فإن دقتها هي 32%

2. لماذا هذا المقياس سيء لعملية التدريب؟

■ **التغيير الطفيف لا يؤثر:** تخيل أنك تريد تحسين الشبكة. ستقوم بإجراء تعديل طفيف جداً على أحد الأوزان (Parameters) في الشبكة. السؤال هو: كيف سيؤثر هذا التعديل الطفيف على الدقة؟ في معظم الحالات، لن يؤثر على الإطلاق
ستظل الدقة 32% لماذا؟

لأن هذا التعديل الطفيف لم يكن كافياً لتغيير إجابة الشبكة من "خطأ" إلى "صواب" لأي من الصور الـ 68 التي أخطأت فيها.

■ **المشتق يساوي صفراً (Derivative = 0):** المشتق يقيس مدى تغير المخرجات عند تغير بسيط في المدخلات. بما أن الدقة لم تتغير (ظلت 32%) رغم تغير الوزن، فإن معدل التغير (المشتق) هو صفر.

■ **النتيجة الكارثية:** عملية تدريب الشبكة العصبية تعتمد كلياً على المشتق. إنها تنظر إلى قيمة المشتق لتقرر:

1- **الاتجاه:** هل يجب زيادة الوزن أم تقليله؟

2- **الحجم:** ما حجم الخطوة التي يجب اتخاذها؟ إذا كان المشتق دائماً صفراً، فكأنك تقول للشبكة: أنت في مكان مثالي، لا تحرك أي شيء. أو كأنك تحاول النزول من جبل في ضباب كثيف، وكل ما يخبرك به **بوصلك** هو لا تتحرك. الشبكة تتوقف عن التعلم تماماً.

3- طبيعة التغير غير المتصلة (Discontinuous):

حتى لو كان التعديل على الوزن كبيراً بما يكفي لتحسين دقة الشبكة، فإن التغير سيكون مفاجئاً وقفزياً. سنتقل الدقة من 32% إلى 33% مباشرة. لا يوجد شيء مثل 32.1% أو 32.5%. هذا الطابع المتقطع يجعل من المستحيل حساب مشتق ذات معنى في معظم النقاط.

الحل: استخدام دالة الخسارة (Loss Function) كمقياس

ما هي دالة الخسارة؟

هي مقياس أكثر ذكاءً ودقة. بدلاً من أن تقول فقط "صواب" أو "خطأ"، فهي تعطي درجة لمدى "خطأ" إجابة الشبكة. كلما كانت الإجابة أبعد عن الإجابة الصحيحة، كانت قيمة الخسارة أعلى.

مثال: قد تكون قيمة الخسارة الحالية هي 0.92543....

لماذا هذا المقياس جيد لعملية "التدريب"؟

- التغيير الطفيف يؤثر: إذا قمنا بإجراء تعديل طفيف على أحد الأوزان، فستتغير قيمة الخسارة أيضاً بشكل طفيف ومستمر. قد تصبح 0.93432... أو 0.91887....
- المشتق لا يساوي صفراً: (Derivative $\neq 0$) بما أن قيمة الخسارة تتغير بشكل مستمر مع تغير الأوزان، فإن المشتق سيكون دائماً قيمة غير صفرية (أو قيمة قريبة من الصفر ولكنها ليست صفراً تماماً).
- النتيجة المثالية: هذا المشتق غير الصفري يعمل مثل بوصلة دقيقة. هو يخبر الشبكة:
 - الاتجاه: إشارة المشتق موجبة، يجب أن تقلل الوزن قليلاً لتقليل الخسارة.
 - الحجم: قيمة المشتق كبيرة، وهذا يعني أن تأثير هذا الوزن كبير، لذا اتخذ خطوة كبيرة.

دالة الخطوة (Step Function) مقابل دالة السيغمويد (Sigmoid Function)

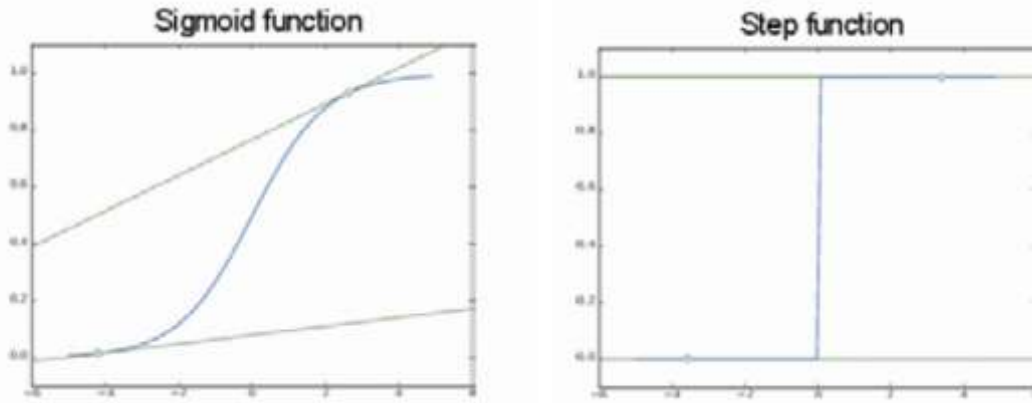
لنتحدث قليلاً عن الفرق بين الدوال المتقطعة والدوال المستمرة.

دالة الخطوة - (Step Function) مثل دقة التعرف:

- سلوكها: هي إما 0 أو 1. لا يوجد وسط. إنها مثل مفتاح الكهرباء، إما مطفأ أو مضيء.
- مشتقتها: مشتقتها يساوي صفراً في كل مكان تقريباً. لماذا؟ لأنك إذا غيرت المدخلات قليلاً، فلن يتغير المخرج (سيظل 0 أو 1). فقط عند نقطة الانتقال تماماً، يكون المشتق غير معرف. هذا يشبه تماماً مشكلة دقة التعرف.
- النتيجة: إذا استخدمت دالة الخطوة كدالة تنشيط في الشبكة العصبية، فلن تتمكن الشبكة من التعلم لأن إشارة التدرج (المشتق) ستضيع وتصبح صفراً.

دالة سيغمويد - (Sigmoid Function) مثل "دالة الخسارة":

- سلوكها: هي منحنى سلس إنها تنتقل بشكل تدريجي ومنتظم من 0 إلى 1.
- مشتقتها: بما أن المنحنى سلس ومستمر، فإن مشتقتها (الذي يمثل ميل المماس عند أي نقطة) هي أيضاً دالة سلسلة وقيمتها ليست صفراً في معظم الأماكن.
- النتيجة: هذا يضمن أنه بغض النظر عن مدى صغر التغيير في المدخلات (الأوزان)، سيكون هناك دائماً تغيير متناسب في المخرجات، وبالتالي ستكون هناك دائماً إشارة تدرج (مشتق) غير صفرية لتوجيه عملية التعلم، هذا هو السبب الأساسي الذي يجعل الشبكات العصبية قادرة على التعلم.



الخاصية	دقة التعرف (RECOGNITION ACCURACY)	دالة الخسارة (LOSS FUNCTION)
طبيعة المخرجات	متقطعة (Discrete) 32% 33% 34%	مستمرة (continuous) (0.925, 0.934, 0.918)
التأثير بالتغيير الطفيف	لا يتغير في معظم الأحيان	يتغير بشكل متناسب
قيمة المشتقة	تساوي صفر في معظم الأماكن	ليست صفر (تقريباً) في كل مكان
إشارة التعلم	معدومة (الشبكة تتوقف عن التعلم)	قوية وواضحة (توجه الشبكة للتحسن)
التشبيه	دالة الخطوة (step function)	دالة سيجمويد (sigmoid function)

Derivative

هي : قياس معدل التغير اللحظي.
تخيل أنك تقود سيارة على طريق جبلي، السرعة التي يظهرها عداد السرعة لديك الآن هي مشتق المسافة التي قطعتها بالنسبة للزمن. إنها لا تخبرك بمتوسط سرعتك في الرحلة بأكملها، بل تخبرك بسرعتك في هذه اللحظة بالذات.
المشتق يُشير إلى مقدار التغير في لحظة معينة
هذه هي الفكرة العامة. بدلاً من معرفة متوسط التغير بين نقطتين، نريد أن نعرف كيف يتغير الشيء بالضبط في نقطة واحدة.

القانون :

$$\frac{df(x)}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

1- الجزء الأول $[f(x+h) - f(x)] / h$: هذا هو معدل التغير المتوسط

- $f(x)$: هي قيمة الدالة عند نقطة ما (لنقل النقطة أ).
 - $f(x+h)$: هي قيمة الدالة عند نقطة أخرى قريبة جداً منها (لنقل النقطة ب)
 - h هو المسافة الصغيرة جداً على محور x بين النقطة أ والنقطة ب.
 - $f(x+h) - f(x)$: هذا هو التغير في قيمة الدالة
 - h : هذا هو التغير في قيمة x
 - -2 الجزء الثاني : $\lim_{h \rightarrow 0}$
- $h \rightarrow 0$ تعني أننا نجعل المسافة h بين النقطتين (أ) و (ب) تقترب من الصفر بشكل لا نهائي

لماذا نجعل h الفارق الصغير في x يقترب من الصفر بدلاً من أن تكون قيمة كبيرة؟

الجواب يكمن في التمييز بين شيئين:

- معدل التغير المتوسط (Average Rate of Change) وهذا ما نحصل عليه عندما تكون h كبيرة.
- معدل التغير اللحظي (Instantaneous Rate of Change) وهذا هو هدفنا من المشتق، ونحصل عليه عندما تقترب h من الصفر.

لنشرح ذلك

مثال: سيارة وعداد السرعة

تخيل أنك تقود سيارة من مدينة (أ) إلى مدينة (ب).

- إذا كانت h كبيرة: هذا يعني أننا ننظر إلى الفترة الزمنية بأكملها، مثلاً 3 ساعات، إذا قطعت 300 كم في 3 ساعات، فإن معدل التغير المتوسط (متوسط السرعة) هو 100 كم/ساعة. هذا الرقم مفيد، لكنه لا يخبرنا أي شيء عن سرعتك الآن أو في أي لحظة معينة خلال الرحلة. هل كنت تسير بسرعة 140 على الطريق السريع ثم تباطأت إلى 60 في المدينة؟ متوسط السرعة لا يخبرنا بهذا التفصيل الدقيق.
- إذا كانت h تقترب من الصفر ($h \rightarrow 0$): هذا يعني أننا ننظر إلى فترة زمنية صغيرة جداً جداً، أصغر من الثانية الواحدة. ماذا يقرأ عداد السرعة في سيارتك هذه اللحظة بالذات؟ هل هو 95، أم 100، أم 102 كم/ساعة؟ هذه القراءة هي السرعة اللحظية. هذا هو بالضبط ما يقيسه المشتق.

الخلاصة :

نحن نريد أن نعرف ميل المنحني أو سرعة التغير عند نقطة محددة، وليس المتوسط على مسافة طويلة .

لدينا نوعان من الاشتقاق :

الاشتقاق العددي (Numerical Differentiation) :

- بما أن دوال الشبكات العصبونية معقدة جداً، لا يمكننا إيجاد مشتقتها تحليلياً بسهولة. لذا نستخدم تقريباً عددياً :
- $$f'(x) \approx (f(x+h) - f(x-h)) / (2h)$$
- الشرح: نحسب قيمة الدالة عند نقطة قريبة إلى اليمين $(x+h)$ ونقطة قريبة إلى اليسار $(x-h)$ ، ونقسم الفرق على المسافة بينهما. هذا يعطينا تقديراً جيداً للانحدار عند x .

الاشتقاق الجزئي (Partial Derivative) :

- عندما تكون الدالة بها عدة متغيرات مثل $f(x_0, x_1)$ ، نحتاج إلى معرفة كيف تتغير الدالة بالنسبة لكل متغير على حدة.
- التعريف: الاشتقاق الجزئي بالنسبة لمتغير معين مثل x_0 هو مشتقة الدالة مع اعتبار جميع المتغيرات الأخرى (x_1) كثوابت.
- المثال $f(x_0, x_1) = (x_0)^2 + (x_1)^2$: الاشتقاق الجزئي بالنسبة لـ x_0 هو $2x_0$ ، وبالنسبة لـ x_1 هو $2x_1$

اشتقاق دالة ما بشكل عددي (Numerical Differentiation) .

الهدف الأساسي:

حساب معدل تغير دالة ما (أي مشتقتها) عند نقطة محددة، باستخدام معادلة رياضية بدلاً من الطرق التحليلية التقليدية.

الطريقة الاولى المتبعة (التنفيذ السيء) :

```

19 # Bad implementation sample
20 def numerical_diff(f, x):
21     h = 10e-50
22     return (f(x+h)- f(x)) / h
23
24 print( np.float32(1e-50))

```

PROBLEMS 13

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

0.0

الدالة (numerical_diff): تأخذ مُدخلين :

- F : الدالة التي نريد حساب اشتقاقها.
- X : النقطة التي نريد حساب الاشتقاق عندها.

اختيار قيمة h : قمنا باختيار قيمة h صغيرة جداً ، وهي $10e-50$ (50 صفراً بعد الفاصلة العشرية)

المشكلة في هذا التنفيذ : على الرغم من أن المنطق يبدو سليماً، إلا أنه لدينا مشكلة وهي خطأ التقريب (Rounding Error).

- السبب في اختيار h الصغيرة : مهمتنا جعل h صغيرة قدر الإمكان لأنه، من الناحية الرياضية ، كلما اقتربت h من الصفر، كانت نتيجة الاشتقاق أدق وأقرب إلى القيمة الحقيقية.
- مشكلة الكمبيوتر (خطأ التقريب) : أجهزة الكمبيوتر لديها قدرة محدودة على تمثيل الأرقام العشرية بدقة تسمى دقة الفاصلة العائمة أو floating-point precision عندما تكون القيمة h صغيرة للغاية مثل $10e-50$ ، يصبح من الصعب على الكمبيوتر التمييز بين القيمة x والقيمة $x + h$. بمعنى آخر، يعتبرهما الكمبيوتر قيمة واحدة متطابقة.
- النتيجة الكارثية : إذا اعتبر الكمبيوتر أن $x + h$ تساوي x تماماً، فإن قيمة الدالة $f(x + h)$ (ستكون مساوية لـ $f(x)$) عليه، يصبح البسط في المعادلة $(f(x + h) - f(x))$ يساوي صفراً، مما يجعل نتيجة الاشتقاق بأكملها صفراً، وهو خطأ فادح .

إذا المشكلة :

استخدام قيم صغيرة جداً في الحسابات يسبب مشاكل حقيقية، لأن الكمبيوتر يفقد قدرته على التعامل معها بدقة.

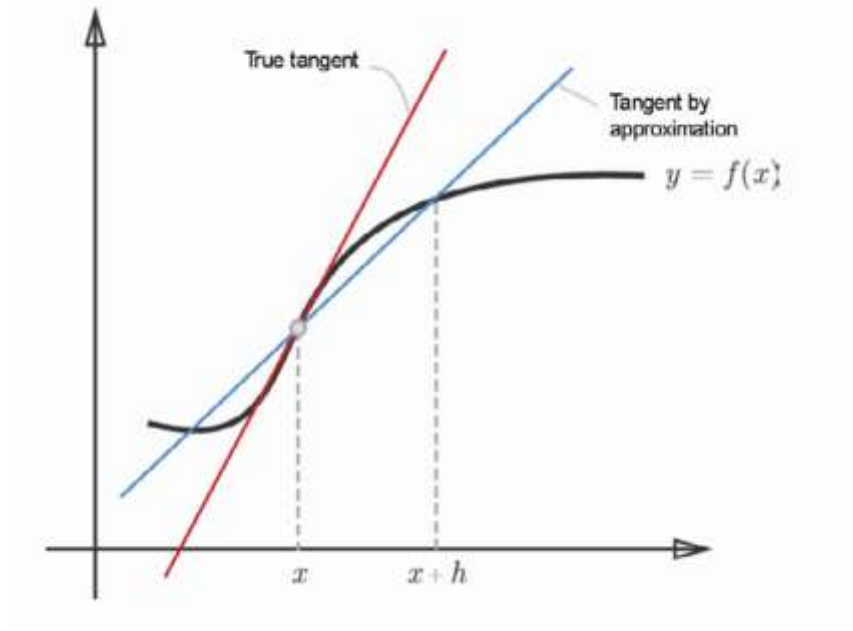
(التحسين) :

بدلاً من استخدام قيمة صغيرة بشكل مبالغ فيه مثل $10e-50$
 الاقتراح: استخدام قيمة (10^{-4}) والتي تساوي 0.0001 كقيمة h . $(1e-4)h$
 لماذا هذه القيمة جيدة؟

- 1- هي صغيرة بما يكفي لتعطي تقديراً جيداً ودقيقاً للمشتق.
- 2- وفي نفس الوقت، هي كبيرة بما يكفي ليتمكن الكمبيوتر من تمثيلها وتخزينها بشكل صحيح دون أن يحولها إلى صفر.

المشكلة في الطريقة الاولى :

الطريقة التي استخدمناها كنا نحسب الفرق في الدالة f بين النقطة x والنقطة التي تليها $x + h$.
 المشكلة هي أن هذا الحساب لا يعطينا المشتق الحقيقي. المشتق الحقيقي هو ميل (انحدار) خط المماس (tangent) الذي يلامس منحنى الدالة عند النقطة x فقط.
 هذا الميل يكون قريباً من ميل المماس، ولكنه ليس مطابقاً له تماماً، وهذا هو مصدر الخطأ (error).
 السبب الجذري لهذا الخطأ هو أننا لا يمكننا جعل قيمة h تساوي صفراً تماماً في الحسابات، لذلك نحن نحصل دائماً على تقدير للمشتق وليس قيمته الحقيقية.

الحل:الفرق المركزي Central Differences :

لتقليل هذا الخطأ وجعل التقدير أقرب ما يمكن للمشتق الحقيقي، نستخدم طريقة أفضل تسمى الفرق المركزي.

طريقة الفرق المركزي بدلاً من أن تقيس الفرق على جانب واحد من النقطة x فقط إلى اليمين في $(x+h)$ ، تقوم بقياس الفرق على جانبي النقطة x (عند $x+h$ وعند $x-h$)

لماذا هذه الطريقة أفضل؟

لأنها تأخذ متوسطاً للميل من جهة اليسار وميل من جهة اليمين، وهذا المتوسط يكون أقرب بكثير لميل المماس الحقيقي عند النقطة x نفسها. إنها تركز الحساب حول النقطة x

```
19 def numerical_diff(f, x):
20     h = 1e-4 # 0.0001
21     return (f(x+h)- f(x-h)) / (2*h)
```

▪ $f(x+h) - f(x-h)$: هذا هو جوهر التحسين الثاني. نحن نحسب قيمة الدالة عند $x+h$ ونطرح منها قيمتها عند $x-h$.

▪ $(2*h)$: نقسم على $2*h$ لأن المسافة الأفقية الكلية بين النقطتين $x+h$ و $x-h$ هي $2h$

التفاضل العددي (Numerical Differentiation): حساب المشتق باستخدام قيم عددية صغيرة جداً

(h) لتقريب النتيجة، إنها طريقة حاسوبية.

الحل التحليلي (Analytical Solution): هو الطريقة الرياضية الدقيقة لإيجاد المشتق باستخدام قواعد

التفاضل (مثل قاعدة القوة، قاعدة الضرب، إلخ).

على سبيل المثال، إذا كانت الدالة $f(x) = x^2$ ، فإن الحل التحليلي يخبرنا أن مشتقتها هي $f'(x) = 2x$ بالضبط، بدون أي خطأ تقريبي.

أمثلة عن التفاضل العددي :

▪ مثال 1 :

دالة تربيعية بسيطة (Quadratic Function) إذا قمنا برسمها، سنحصل على شكل قطع مكافئ (Parabola) مفتوح للأعلى

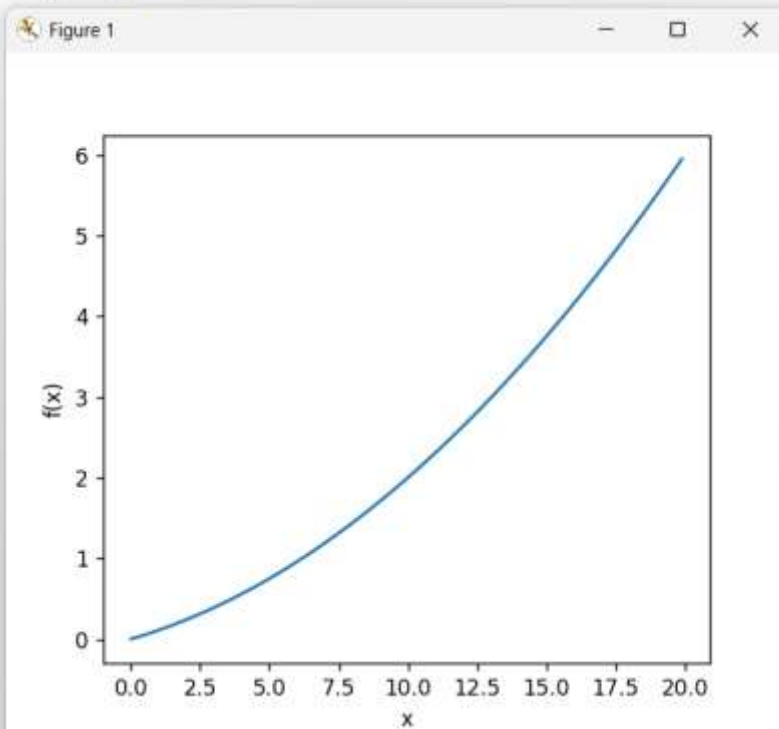
```
23 # y = 0.01x^2 + 0.1x
24 def function_1(x):
25     return 0.01*x**2 + 0.1*x
```

1- رسم الدالة بيانياً (Plotting the Function) :

```

62 x = np.arange(0.0, 20.0, 0.1)
63 y = function_1(x)
64 plt.xlabel("x")
65 plt.ylabel("f(x)")
66 plt.plot(x, y)
67 plt.show()
68

```



قبل حساب المشتق، من المفيد جداً أن نرى شكل الدالة. الكود التالي يقوم برسمها

▪ `x = np.arange(0.0, 20.0, 0.1)`

إنشاء مصفوفة من الأرقام (x) تبدأ من 0 وتنتهي عند 20، وبزيادة قدرها 0.1

▪ `y = function_1(x)`

حساب قيم y المقابلة لكل قيمة في مصفوفة x

▪ `plt.xlabel("x")` `plt.ylabel("f(x)")`

إضافة عناوين للمحورين

▪ `plt.plot(x, y)` : رسم المخطط البياني

▪ `plt.show()` : عرض المخطط على الشاشة

الرسم البياني يوضح شكل الدالة $y = 0.01x^2 + 0.1x$ على النطاق من $x = 0$ إلى (منحني يتصاعد بشكل تدريجي)

2- الان سنستخدم دالة `numerical_diff` لحساب ميل المنحني عند نقطتين مختلفتين

```

36
37 print('EX1: ', numerical_diff(function_1, 5))
38 print('EX2: ', numerical_diff(function_1, 10))

```

PROBLEMS 13 OUTPUT DEBUG CONSOLE TERMINAL PORTS

EX1: 0.1999999999990898

EX2: 0.2999999999986347

المشتق الذي حسبناه هو مقدار التغير في دالة $f(x)$ بالنسبة لمتغير x

وهذا المقدار يمثل تمامًا انحدار أو ميل الدالة عند تلك النقطة

بعبارة أخرى، عندما حسبنا `numerical_diff(function_1, 5)`، كنا نحسب بالضبط ميل الخط المماس لمنحني

الدالة عند النقطة التي يكون فيها $x=5$

3- المقارنة مع الحل التحليلي

لتقييم مدى دقة طريقتنا العددية، نقارنها بالحل التحليلي (Analytical Solution)

بالنسبة لدالتنا $f(x) = 0.01x^2 + 0.1x$ ، المشتق التحليلي هو $f'(x) = 0.02x + 0.1$

باستخدام هذه المعادلة، يمكننا حساب القيم الحقيقية للمشتق:

عندما $x = 5$ ، المشتق الحقيقي هو $0.02 * 5 + 0.1 = 0.2$

عندما $x = 10$ ، المشتق الحقيقي هو $0.02 * 10 + 0.1 = 0.3$

4- دقة التفاضل العددي

الآن نقارن هذه القيم "الحقيقية" بالنتائج التي حصلنا عليها من التفاضل العددي

النتائج ليست متطابقة تمامًا، ولكن الخطأ صغير جدًا .

هذا يعني أن القيمة التي أعطاها الكمبيوتر (0.2999999.....) هي قريبة جدًا من القيمة الحقيقية (0.2) لدرجة أنه

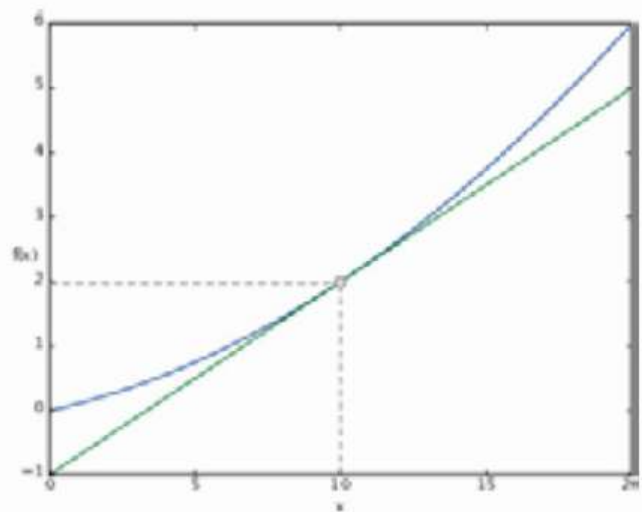
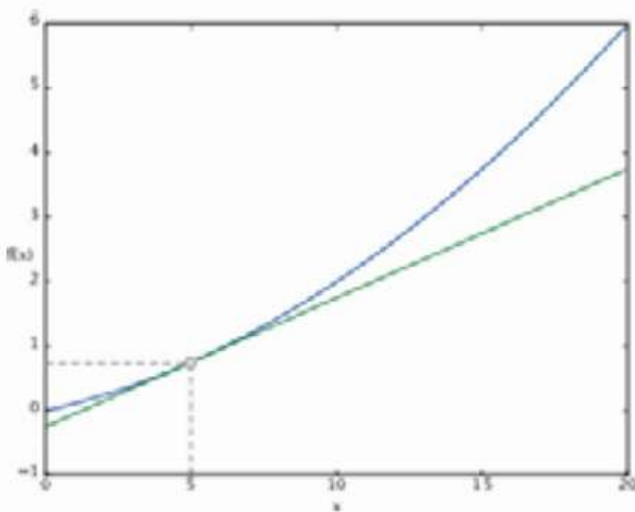
في معظم التطبيقات العملية، يمكننا اعتبارها نفس القيمة.

5- رسم الخطوط المماسية

سنستخدم قيم الميل التي حصلنا عليها من التفاضل العددي (0.2 و 0.3) لرسم رسوم بيانية لخطوط ميلها يساوي

قيم التفاضل العددي

نرى ان المشتقات تتوافق مع المماسات للدالة



ننتقل من عالم الدوال ذات المتغير الواحد مثل $(f(x))$ إلى عالم الدوال ذات المتغيرات المتعددة مثل $(f(x_0, x_1))$

وهو أساس مفهوم المشتق الجزئي (Partial Derivative)

1. الانتقال إلى الدوال متعددة المتغيرات

- الدالة الجديدة: $f(x_0, x_1) = x_0^2 + x_1^2$
- الفرق الأساسي: على عكس الدالة $y = 0.01x^2 + 0.1x$ السابقة التي كانت تأخذ مدخلاً واحداً (x) وتعطي مخرجاً واحداً (y) ، هذه الدالة الجديدة تأخذ مدخلين $(x_0$ و $x_1)$ وتعطي مخرجاً واحداً.
- مثال: إذا كان $x_0 = 2$ و $x_1 = 3$ ، فإن قيمة الدالة هي:
- $f(2, 3) = 2^2 + 3^2 = 4 + 9 = 13$
- التمثيل الهندسي: بما أن للدالة مدخلين، فإن رسمها البياني سيكون سطحاً ثلاثي الأبعاد، وليس مجرد خط منحنٍ على مستوى ثنائي الأبعاد. شكل هذه الدالة المحددة يشبه وعاءاً أو قبة (Paraboloid)، حيث تكون قيمته صفر عند النقطة $(0, 0)$ ويزداد كلما ابتعدنا عن هذه النقطة في أي اتجاه.

2. مفهوم المشتق الجزئي (Partial Derivative)

هي الطريقة التي نحسب بها الميل أو معدل التغير في دالة متعددة المتغيرات.

- المشكلة:** في دالة ثلاثية الأبعاد، لا يوجد "ميل" واحد فقط عند نقطة ما. الميل يعتمد على الاتجاه الذي تتحرك فيه على السطح. هل تتحرك على طول محور x_0 ؟ أم على طول محور x_1 ؟ أم في اتجاه قطري بينهما؟
- الحل:** المشتق الجزئي يحل هذه المشكلة بالنظر إلى اتجاه واحد في كل مرة.
- المشتق الجزئي بالنسبة لـ x_0 :** $\left(\frac{\partial f}{\partial x_0}\right)_{x_0}$: نحن نحسب ميل السطح في اتجاه محور x_0 فقط. للقيام بذلك، نعتبر المتغير الآخر (x_1) ثابتاً (كأنه رقم ثابت) ونشتق الدالة بالنسبة لـ x_0 فقط.
- المشتق الجزئي بالنسبة لـ x_1 :** $\left(\frac{\partial f}{\partial x_1}\right)_{x_1}$: نحن نحسب ميل السطح في اتجاه محور x_1 فقط. هنا، نعتبر x_0 ثابتاً ونشتق الدالة بالنسبة لـ x_1 .

```
41 def function_2(x):
42     return x[0]**2 + x[1]**2
43 # or simply: return np.sum(x**2)
```

الشرح:

• المدخل: x مصفوفة تحتوي على عنصرين.

• $x[0]$: يمثل المتغير الأول x_0

• $x[1]$: يمثل المتغير الثاني x_1

▪ البديل الأبسط `return np.sum(x**2)`

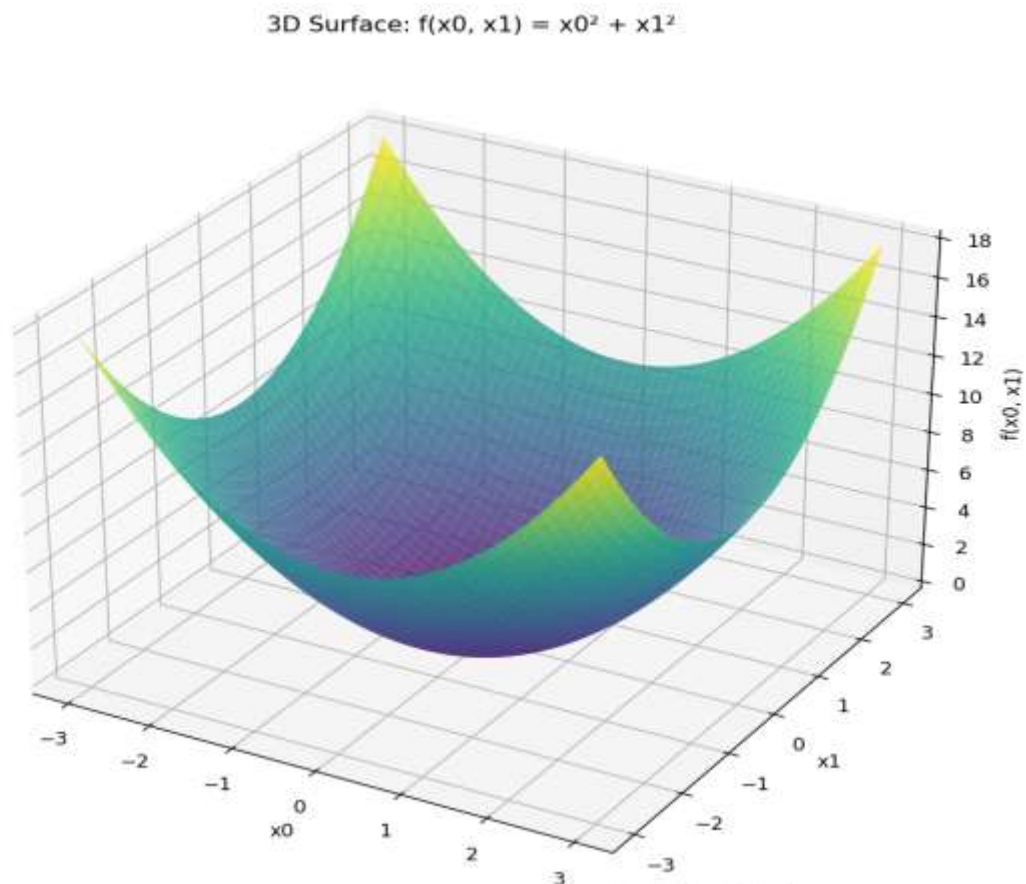
▪ هذا السطر يقوم بنفس العمل بشكل أكثر كفاءة `x**2` يقوم بتربيع كل عنصر في المصفوفة x ، و `np.sum` يقوم بجمع كل العناصر المربعة.

1- رسم الدالة ثلاثية الأبعاد :

```

71 x0 = np.linspace(-3, 3, 100)
72 x1 = np.linspace(-3, 3, 100)
73 X0, X1 = np.meshgrid(x0, x1)
74
75 Y = function_2(np.array([X0, X1]))
76
77 fig = plt.figure(figsize=(10, 5))
78 ax1 = fig.add_subplot(1, 2, 2, projection='3d')
79 ax1.plot_surface(X0, X1, Y, cmap='viridis', edgecolor='none', alpha=0.8)
80 ax1.set_xlabel('x0')
81 ax1.set_ylabel('x1')
82 ax1.set_zlabel('f(x0, x1)')
83 ax1.set_title('3D Surface: f(x0, x1) = x02 + x12')

```



▪ `np.linspace(-3, 3, 100)` :

• ينشئ مصفوفة من 100 نقطة موزعة بالتساوي من -3 إلى 3.

▪ `np.meshgrid(x0, x1)` :

• تأخذ المصفوفتين أحاديّتي البعد x_0 و x_1 وتنشئ منهما مصفوفتين ثنائيّتي البعد x_0 و x_1 .

• تحتوي على إحداثيات المحور الأفقي لكل نقطة في الشبكة، و x_1 تحتوي على إحداثيات المحور العمودي.

• معاً، تشكّلان شبكة إحداثيات كاملة.

▪ `Y = function_2(np.array([X0, X1]))` :

• هنا يتم حساب قيمة الدالة $f(x_0, x_1)$ لكل نقطة في الشبكة التي أنشأناها. الناتج Y هو أيضاً مصفوفة ثنائية الأبعاد تحتوي على ارتفاع السطح فوق كل نقطة (x_0, x_1)

▪ `ax1.plot_surface(x0, x1, Y)` :

• هذه هي الدالة التي ترسم السطح الفعلي باستخدام المصفوفات الثلاث (x_0, x_1, Y) كإحداثيات (x, y, z) لكل نقطة على السطح.

2- تعميم :

المشتق الجزئي هو طريقة لقياس معدل التغير (أو الميل) لدالة متعددة المتغيرات مع التركيز على متغير واحد فقط في كل مرة، مع اعتبار جميع المتغيرات الأخرى ثوابت (أي قيمها ثابتة ولا تتغير)

▪ **المشتق الجزئي بالنسبة لـ x_0** (تكتب رياضياً $\frac{\partial f}{\partial x_0}$) : تقيس مدى تغير الدالة f عند تغير x_0 طالما أن x_1 ثابت.

▪ **المشتقة الجزئية بالنسبة لـ x_1** (تكتب رياضياً $\frac{\partial f}{\partial x_1}$) : تقيس مدى تغير الدالة f عند تغير x_1 طالما أن x_0 ثابت.

المسألة العملية (Question 1) :

احسب المشتق الجزئي $f(x_0, x_1) = x_0^2 + x_1^2$ للدالة بالنسبة للمتغير x_0 عند النقطة التي يكون فيها $x_0 = 3$ و $x_1 = 4$

استراتيجية الحل :

هنا يكمن جوهر الشرح. الدالة التي أنشأناها سابقاً لحساب التفاضل العددي `numerical_diff(f, x)` مصممة للعمل مع دوال ذات متغير واحد فقط. لا يمكننا أن نعطيها الدالة `function_2(x)` مباشرة لأنها تأخذ متغيرين.

الحل الذكي هو "خدعة" برمجية بسيطة :

1. إنشاء دالة مؤقتة جديدة : نقوم بتعريف دالة جديدة ذات متغير واحد فقط. هذه الدالة الجديدة تخفي أو تثبت قيمة المتغير الآخر.

2. تطبيق التفاضل العددي : نستخدم دالة `numer_diff` المألوفة على هذه الدالة المؤقتة الجديدة.

تطبيق الحل على المسألة:

الخطوة 1 :

تثبيت المتغير x_1 نحن نريد حساب المشتق بالنسبة لـ x_0 عند النقطة التي يكون فيها $x_1 = 4$
إذا، سنقوم بإنشاء دالة جديدة يكون فيها x_1 قيمة ثابتة تساوي 4

```
55 def function_tmp1(x0):
56     return x0*x0 + 4.0**2.0
```

شرح الكود:

- 1- هذه الدالة الجديدة اسمها function_tmp1 (دالة مؤقتة 1)
- 2- هي تأخذ متغيراً واحداً فقط وهو x_0
- 3- عند حسابها، فإنها تربع x_0 وتضيف إليه قيمة 4.0×2.0 (وهي 4 تربيع، أي 16) هذه القيمة "16" هي في الحقيقة قيمة x_1^2 بعد تثبيت x_1 عند 4
- 4- بعبارة أخرى، لقد حولنا مشكلة الاشتقاق الجزئي إلى مشكلة اشتقاق عادي لدالة من متغير واحد :
- 5- $g(x_0) = x_0^2 + 16$

الخطوة 2:

حساب التفاضل العددي للدالة المؤقتة الآن يمكننا بسهولة استخدام دالة numerical_diff التي نعرفها على هذه الدالة المؤقتة function_tmp1 عند النقطة $x_0 = 3$

```
45 print('EX1: ', numerical_diff(function_tmp1, 3.0))
```

PROBLEMS 13 OUTPUT DEBUG CONSOLE TERMINAL PORTS

EX1: 6.000000000000378

الشرح:

النتيجة والمقارنة بالحل التحليلي :

- النتيجة العددية: يعطينا البرنامج النتيجة 6.000000000000378
- التحقق من الصحة (الحل التحليلي): كيف نعرف أن هذه الإجابة صحيحة؟
- المشتق الجزئي لـ $f(x_0, x_1) = x_0^2 + x_1^2$ بالنسبة لـ x_0 هي $(2x_0)$ نعامل (x_1^2) كأنه ثابت ومشتقته يساوي صفراً
- عند التعويض بقيمة $x_0 = 3$ ، نحصل على $2 * 3 = 6$
- المقارنة بين النتيجة التحليلية الدقيقة 6 والنتيجة العددية 6.000000000000378 تظهر أن الخطأ ضئيل جداً ويمكن اعتباره مطابقاً تقريباً. هذا يثبت أن طريقتنا تعمل بشكل صحيح

الخلاصة :

لحساب المشتق الجزئي عددياً لدالة متعددة المتغيرات، نتبع الخطوات التالية:

1. نحدد المتغير الذي نريد حساب المشتقة بالنسبة له.
2. نثبت جميع المتغيرات الأخرى عند قيمها المطلوبة.
3. ننشئ دالة مؤقتة جديدة ذات متغير واحد فقط تعكس هذه الحالة (المتغير المطلوب هو المتغير الوحيد، وبقية المتغيرات هي قيم ثابتة).
4. نطبق دالة التفاضل العددي العادية (numerical_diff) على هذه الدالة المؤقتة الجديدة.

Gradient

بعد أن تعلمنا في الفقرة السابقة كيفية حساب المشتق الجزئي لكل متغير على حدة (مثل حساب المشتق بالنسبة لـ x_0 ثم حسابها بالنسبة لـ x_1)

يظهر سؤال منطقي :

ماذا لو أردنا جمع كل هذه المعلومات في مكان واحد؟ ماذا لو أردنا أن نحصل على "صورة كاملة" عن تغير الدالة في جميع الاتجاهات في نفس الوقت؟

هنا يأتي دور مفهوم التدرج (Gradient) :

التدرج ليس مجرد رقم، بل هو متجه (Vector) يجمع كل المشتقات الجزئية لدالة ما في كيان واحد. هذا المتجه يخبرنا شيئاً مهماً جداً في نفس الوقت :

1. **الاتجاه**: يشير إلى الاتجاه الذي تزداد فيه الدالة بأقصى سرعة ممكنة.
 2. **المقدار**: طول هذا المتجه يخبرنا عن مدى سرعة أو "شدة" هذا الزيادة.
- تخيل أنك تقف على جبل (وهو يمثل دالتنا ثلاثية الأبعاد $f(x_0, x_1)$):
- المشتق الجزئي بالنسبة لـ x_0 يخبرك بانحدار الجبل في اتجاه الشرق.
 - والمشتق الجزئي بالنسبة لـ x_1 يخبرك بانحداره في اتجاه الشمال.
 - التدرج (Gradient) هو الذي يعطيك **البوصلة الكاملة** التي تشير مباشرة إلى الاتجاه الذي يجب أن تسير فيه لتصل إلى القمة بأسرع طريق ممكن، ويخبرك أيضاً بمدى انحدار هذا الطريق

```

50 def _numerical_gradient_no_batch(f, x):
51     h = 1e-4 # 0.0001
52     grad = np.zeros_like(x)
53     for idx in range(x.size):
54         tmp_val = x[idx]
55         x[idx] = float(tmp_val) + h
56         fxh1 = f(x) # f(x+h)
57         x[idx] = tmp_val - h
58         fxh2 = f(x) # f(x-h)
59         grad[idx] = (fxh1 - fxh2) / (2*h)
60     return grad
61 def numerical_gradient(f, X):
62     if X.ndim == 1:
63         return _numerical_gradient_no_batch(f, X)
64     else:
65         grad = np.zeros_like(X)
66         for idx, x in enumerate(X):
67             grad[idx] = _numerical_gradient_no_batch(f, x)
68         return grad

```

1- الدالة `def _numerical_gradient_no_batch(f, x)` : تقوم بالحساب الفعلي لتدرج نقطة واحدة فقط.

1. `grad = np.zeros_like(x)` :

▪ هنا ننشئ مصفوفة (متجه) فارغة بنفس حجم المدخل x ، وكل عناصرها أصفار. هذه المصفوفة ستكون "حاوية" النتائج، حيث سنخزن فيها المشتق الجزئي لكل متغير.

2. `for idx in range(x.size)` :

▪ نبدأ حلقة تكرارية (loop) تمر على كل بُعد في مصفوفة x إذا كانت x تحتوي على x_0 و x_1 ، فستمر مرتين:

مرة لـ x_0 عندما يكون $idx = 0$ ومرة لـ x_1 عندما يكون $idx = 1$

3. `tmp_val = x[idx]` :

▪ قبل أن نغير أي شيء، نحفظ القيمة الأصلية للمتغير الحالي في متغير مؤقت.

4. حساب `f(x+h)` :

▪ نأخذ قيمة المتغير الحالي ونزيدها قليلاً $(+h)$ ، ثم نحسب قيمة الدالة الكاملة بهذه القيمة الجديدة.

5. حساب `f(x-h)` :

▪ نرجع قيمة المتغير الحالية إلى أصلها، ثم ننقصها قليلاً $(-h)$ ، ونحسب قيمة الدالة مرة أخرى.

6. `grad[idx] = (fxh1 - fxh2) / (2*h)` :

▪ هنا نستخدم صيغة الفرق المركزي التي تعلمناها سابقاً لحساب المشتق الجزئي للمتغير الحالي ($x[idx]$)

النتيجة نخزنها في المكان المناسب داخل متجه النتائج `grad`

7. `x[idx] = tmp_val` :

- بعد أن انتهينا من حساب المشتق للمتغير الحالي، نعيد قيمته إلى ما كانت عليه تماماً. هذا يضمن أن الحساب التالي في الحلقة (للمتغير التالي) لن يتأثر بأي تغييرات قمناها سابقاً.
- 2- الدالة `def numerical_gradient(f, X)` : هذه الدالة تعمل كـ "مدير" أو "غلاف" للدالة الأساسية، وتجعلها أكثر ذكاءً وقوة.

هذه الدالة ذكية لأنها تتحقق من شكل المدخل X :

- الحالة الأولى ($X.ndim == 1$) :

- إذا كان المدخل عبارة عن مصفوفة ذات بعد واحد (أي يمثل نقطة واحدة مثل $[3, 4]$)، فإنها تستدعي الدالة الأساسية `numerical_gradient_no_batch` مباشرة وتعطيها النقطة.

- الحالة الثانية (else) :

- إذا كان المدخل عبارة عن مصفوفة ذات بعدين (أي يمثل مجموعة من النقاط ، مثل $[[1, 2], [3, 4], [5, 6]]$)، فإنها تقوم بالآتي:

- تنشئ حاوية نتائج كبيرة `grad`
- تمر على كل نقطة x داخل المجموعة X
- لكل نقطة، تستدعي الدالة الأساسية لحساب تدرج تلك النقطة وحدها.
- تخزن نتيجة تدرج كل نقطة في حاوية النتائج الكبيرة.

الخلاصة :

باختصار، الكود يقوم بأتمتة عملية حساب جميع المشتقات الجزئية وتجميعها في متجه واحد يسمى التدرج. هذا المتجه هو أداة قوية تعطينا صورة كاملة عن اتجاه وقوة تغير الدالة عند نقطة معينة، وهو المفتاح الأساسي لخوارزميات التحسين مثل "الانحدار التدريجي (Gradient Descent)" التي تُستخدم لتدريب الشبكات العصبية

الآن لنحسب التدرج عند نقطة الاختبار الأولى التي يكون فيها $x_0 = 3$ و $x_1 = 4$:

```
70 print('EX1: ', numerical_gradient(function_2, np.array([3.0, 4.0])))
```

PROBLEMS 13 OUTPUT DEBUG CONSOLE TERMINAL PORTS

EX1: [6. 8.]

شرح الخرج :

- الرقم الأول 6 هو المشتق الجزئي بالنسبة لـ x_0 هذا يعني أنك لو تحركت خطوة صغيرة في اتجاه x_0 فقط (بافتراض أن x_1 ثابت)، فإن ارتفاعك سيزداد بمقدار 6 أضعاف طول خطوتك.
- الرقم الثاني 8 هو المشتق الجزئي بالنسبة لـ x_1 هذا يعني أنك لو تحركت خطوة صغيرة في اتجاه x_1 فقط، فإن ارتفاعك سيزداد بمقدار 8 أضعاف خطوتك.
- الخلاصة: التدرج (6, 8) يجمع هذين الميلين في متجه واحد، ويشير إلى الاتجاه الذي يحقق لك أقصى استفادة من كلا الميلين.

نقطة الاختبار الثانية التي يكون فيها $x_0 = 0$ و $x_1 = 2$:

```
70 print('EX2: ', numerical_gradient(function_2, np.array([0.0, 2.0])) )
```

PROBLEMS 13 OUTPUT DEBUG CONSOLE TERMINAL PORTS

EX2: [0. 4.]

شرح الخرج :

- التدرج هو سهم يشير مباشرة إلى الأعلى في اتجاه محور x_1 فقط. لماذا؟
- لأنك عند النقطة (0, 2) إذا تحركت في اتجاه x_0 (إلى اليمين أو اليسار)، فلن يتغير ارتفاعك كثيراً في البداية (لذلك المشتق الجزئي بالنسبة لـ x_0 هي 0).
- أما إذا تحركت في اتجاه x_1 (لأعلى)، فسيرتفع ارتفاعك بسرعة (لذلك المشتق الجزئي بالنسبة لـ x_1 هي 4).

ملاحظة بسيطة:

- النتيجة الفعلية من الحاسوب [6.00000000000037801, 7.9999999999991189]
- لكن مكتبة NumPy "تجمل" النتيجة وتعرضها لك كـ [6., 8.] لتكون أسهل في القراءة.
- هذا يذكرنا مرة أخرى بأن التفاضل العددي هو تقريب دقيق جداً، وليس قيمة رياضية مثالية، والفروق الصغيرة جداً في النتائج طبيعية.

تخيل معي أن دالتنا $f(x_0, x_1) = x_0^2 + x_1^2$ هي عبارة عن خريطة لتل كبير. كل نقطة (x_0, x_1) على هذه الخريطة لها ارتفاع معين. أنت تقف في مكان ما على هذا التل وتريد أن تعرف أسرع طريق للوصول إلى القمة. ما تعلمناه سابقاً هو أن التدرج (Gradient) هو مثل "بوصلة" عند أي نقطة تقف فيها، تعطيك هذه البوصلة سهماً (متجهاً) يشير مباشرة إلى الاتجاه الذي يصعد فيه التل بأسرع شكل ممكن.

ماذا تخبرنا هذه الأسهم (الأسهم هي التدرجات)؟

عندما ننظر إلى الرسم البياني، سنرى شبكة من الأسهم (السهام) هذه الأسهم تخبرنا شيئاً في نفس الوقت:

- الاتجاه (Direction): كل سهم يشير إلى أسرع طريق صاعد. إذا تتبعنا اتجاهات الأسهم من أي مكان، ستلاحظ أنها كلها تتدفق نحو نقطة واحدة. هذه النقطة هي "القمة" أو "أعلى نقطة" في دالتنا، وهي (0, 0) في هذه الحالة. كل بوصلة تشير إلى القمة الأعلى بالنسبة لموقعها.
- المقدار أو الطول (Magnitude): طول السهم يدل على مدى انحدار التل في تلك النقطة. الأسهم الطويلة = تدرج كبير = انحدار شديد يعني: إذا تحركت خطوة صغيرة في اتجاه السهم، سيتغير ارتفاعك بشكل كبير. الأسهم القصيرة = تدرج صغير = انحدار قليل/لطيف يعني: إذا تحركت خطوة صغيرة في اتجاه السهم، سيتغير ارتفاعك بشكل طفيف في دالتنا: المركز (0, 0) هو أدنى نقطة (أقل ارتفاع) فكلما ابتعدنا عن المركز، يزيد الارتفاع.
- في المركز (0, 0): التدرج = صفر ← لا سهم (أو سهم بطول صفر)
- كلما ابتعدنا عن المركز: التدرج يزيد ← الأسهم تصبح أطول

$$f(x, y) = x^2 + y^2$$

$$\partial f / \partial x = 2x$$

$$\partial f / \partial y = 2y$$

مثال:

عند (2,0): التدرج = (4,0) ← طول = 4 ← سهم أطول
عند (0,0): التدرج = (0,0) ← طول = 0 ← لا يوجد سهم

الخلاصة :

- في دالتنا هذه، كلما ابتعدنا عن المركز (0,0):
- يزيد التدرج (الأسهم تصبح أطول)
- الانحدار يزداد (التل يصبح أكثر حدة)
- التغير في الارتفاع يكون أكبر لكل خطوة

العكس:

كلما اقتربنا من المركز:

- الأسهم تصبح أقصر
- الانحدار يقل
- حتى نصل للمركز حيث الانحدار = صفر

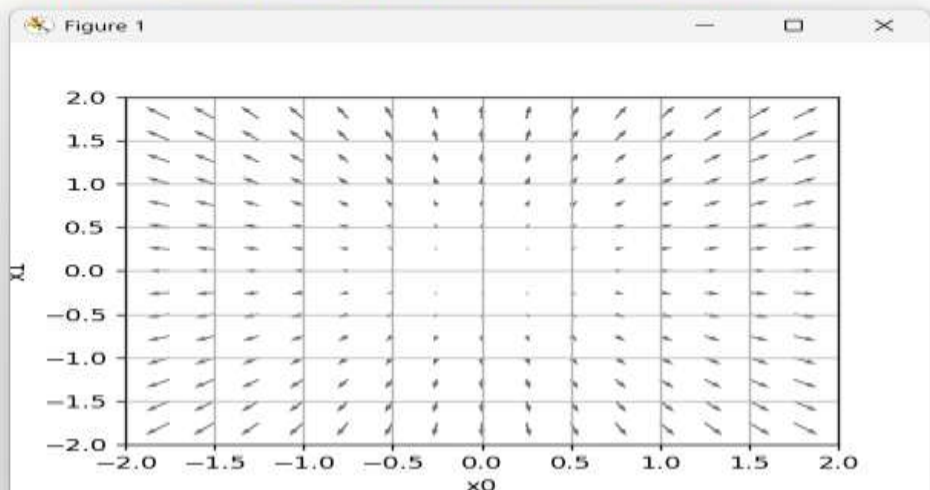
كيف نرسم هذه الخريطة؟

- 1- نختار مجموعة من النقاط على الخريطة (شبكة من النقاط).
- 2- عند كل نقطة، نستخدم دالة numerical_gradient لحساب اتجاه وقوة البوصلة (أي التدرج).
- 3- نرسم سهمًا صغيرًا عند كل نقطة بناءً على هذه النتيجة

```

70 x0 = np.arange(-2, 2.5, 0.25)
71 x1 = np.arange(-2, 2.5, 0.25)
72 X, Y = np.meshgrid(x0, x1)
73 X = X.flatten()
74 Y = Y.flatten()
75 grad = numerical_gradient(function_2, np.array([X, Y]).T).T
76 plt.figure()
77 plt.quiver(X, Y, grad[0], grad[1], angles="xy", color="#666666")
78 plt.xlim([-2, 2])
79 plt.ylim([-2, 2])
80 plt.xlabel('x0')
81 plt.ylabel('x1')
82 plt.grid()
83 plt.draw()
84 plt.show()

```



شرح مفصل ل طريقة التدرج (Gradient Method)

1. المشكلة الأساسية: البحث عن المعاملات المثلى

- في عالم تعلم الآلة، وخاصة عند تدريب الشبكات العصبونية، الهدف دائماً هو العثور على المعاملات (Parameters) المثلى. هذه المعاملات هي الأوزان (Weights) والانحيازات (Biases) داخل الشبكة.
- **ماذا تعني مثلى؟** المعاملات المثلى هي تلك التي تجعل دالة الخسارة (Loss Function) تصل إلى أقل قيمة ممكنة. دالة الخسارة هي ببساطة مقياس لمدى خطأ الشبكة العصبونية في أداء مهمتها. كلما قل الخطأ، كانت الشبكة أفضل.
 - **التحدي:** غالباً ما تكون دالة الخسارة هذه معقدة جداً، ومساحة البحث عن المعاملات (المساحة التي يمكن أن توجد فيها كل الأوزان والانحيازات الممكنة) كبيرة جداً. إن الأمر أشبه بالبحث عن إبرة في كومة قش، أو كأنك تبحث عن أدنى نقطة في منطقة جبلية وعرة وممتدة في ليلة مظلمة. من المستحيل تخمين مكان هذه القيمة الدنيا.

2. الحل: استخدام بوصلة التدرج

- هنا يأتي دور طريقة التدرج (Gradient Method) لقد تعلمنا سابقاً أن التدرج (Gradient) عند أي نقطة هو متجه (سهم) يشير إلى الاتجاه الذي تزداد فيه الدالة بأسرع شكل ممكن.
- **والآن الفكرة العبقريّة:** إذا كان التدرج يشير إلى أسرع طريق للصعود إلى القمة، فماذا لو مشينا في الاتجاه المعاكس تماماً؟ الإجابة: سنكون قد مشينا في أسرع طريق للهبوط نحو القاع!
 - إذا للعثور على أدنى قيمة لدالة الخسارة، كل ما علينا فعله هو:
 1. حساب التدرج (اتجاه الصعود الأسرع).
 2. التحرك خطوة في الاتجاه المعاكس تماماً للتدرج (اتجاه الهبوط الأسرع).

3. خوارزمية العمل:

- طريقة التدرج هي في جوهرها رحلة خطوة بخطوة نحو القاع. إليك كيف تعمل:
1. **اختر نقطة بداية عشوائية:** ابدأ بقيم عشوائية للمعاملات (الأوزان والتحيزات). هذا كأنك تُسقط شخصاً في مكان ما على الخريطة الجبلية.
 2. **احسب التدرج:** عند هذه النقطة، استخدم "البوصلة" (التدرج) لمعرفة اتجاه الصعود الأعلى.
 3. **اتخذ خطوة صغيرة في الاتجاه المعاكس:** تحرك من موقعك الحالي خطوة صغيرة في الاتجاه المعاكس تماماً لما تشير إليه البوصلة.
 4. **كرر العملية:** الآن أصبحت في نقطة جديدة وأكثر انخفاضاً. كرر الخطوة 2: احسب التدرج الجديد في هذا الموقع الجديد، ثم كرر الخطوة 3: اتخذ خطوة أخرى في الاتجاه المعاكس له.
- من خلال تكرار هذه العملية آلاف المرات، فإنك تقلل من قيمة دالة الخسارة تدريجياً، وتنحدر ببطء نحو أدنى نقطة ممكنة في الوادي، مما يجد لك المعاملات المثلى للشبكة العصبونية.

طريقة الانحدار التدريجي (Gradient Descent Method) :

يُستخدم عندما يكون الهدف هو البحث عن القيمة الدنيا (Minimum) هذا هو النوع الأكثر شيوعاً في تعلم الآلة، لأننا عادة ما نريد تقليل الخطأ أو الخسارة .

طريقة الصعود التدريجي (Gradient Ascent Method) :

يُستخدم عندما يكون الهدف هو البحث عن القيمة العليا (Maximum) .

ملاحظة :

يمكن تحويل مسألة البحث عن القيمة العليا إلى مسألة بحث عن القيمة الدنيا ببساطة عن طريق عكس إشارة الدالة (ضربها في -1). لذلك، فإن طريقة الانحدار التدريجي هي الأداة الأساسية التي يمكن استخدامها لحل كلا النوعين من المسائل

معادلة الانحدار التدريجي :

$$x_0 = x_0 - \eta \frac{\partial f}{\partial x_0}$$

$$x_1 = x_1 - \eta \frac{\partial f}{\partial x_1}$$

شرح القانون :

المتغير x_0 أو x_1 : يمثل أحد معاملات الشبكة العصبية التي نحاول تحسينها (مثل الوزن أو الانحياز).

(المشتق الجزئي/التدرج):

- تمثل التدرج (Gradient) ، وهي قيمة المشتقة الجزئية لدالة الخسارة (f) بالنسبة للمتغير x_0 هذا التدرج يحدد اتجاه الانحدار لتقليل (f) و ميل الخسارة عند النقطة الحالية .
- الناقص (-) : يتم استخدام علامة الناقص لأننا نريد أن نتحرك في الاتجاه المعاكس لاتجاه التدرج. إذا كان التدرج موجباً (+)، فهذا يعني أننا نتجه نحو الزيادة، لذا يجب أن نتحرك في الاتجاه السالب لتقليل الخسارة .

(η) معامل التعلم (مقياس خطوتك) إيتا :

هو رقم واحد تحدده أنت قبل بدء التدريب. يسمى معامل التعلم (Learning Rate) الوظيفة :

- يُضبط مقدار التعديل الذي سيتم إجراؤه على المعاملات في كل خطوة تحديث
 - يحدد كم يجب أن تتعلم الشبكة و كم يجب أن يتم تحديث المعاملات بناءً على التدرج المحسوب
- مثال: عدنا إلى متسلك الجبل.

- إذا كان η كبيراً، فالمتسلف سيخطو خطوات طويلة وواثقة.
- إذا كان η صغيراً، فالمتسلف سيخطو خطوات صغيرة وحذرة.

معامل التعلم كبير جداً أم صغير جداً؟

اختيار قيمة η الصحيحة هو أمر حاسم، وإلا فلن نصل إلى الحل الأمثل

1- إذا كان η كبيراً جداً:

المتسلف يخطو خطوات عملاقة. قد يقفز فوق قاع الوادي وينتهي به الأمر على الجانب الآخر، ربما في مكان أعلى مما كان عليه! هذا قد يجعل عملية التدريب غير مستقرة ومتذبذبة

اي : يؤدي إلى خطوات كبيرة نحو القيمة الدنيا، وقد يسبب تذبذباً حول القيمة المثلى أو عدم التقارب على الإطلاق..

2- إذا كان η صغيراً جداً :

- المشكلة : المتسلف يخطو خطوات صغيرة جداً وبطيئة للغاية. نعم، إنه يتحرك في الاتجاه الصحيح،

ولكنه سيحتاج إلى ملايين الخطوات للوصول .

- النتيجة : عملية التدريب ستكون بطيئة بشكل لا يطاق وقد تتوقف قبل الوصول إلى حل جيد لأن الوقت سينفذ.

3- إذا كان η مناسباً (مثلاً 0.01 أو 0.001):

- الحل المثالي : المتسلف يخطو خطوات مدروسة وفعالة. خطواته كبيرة بما يكفي لتحقيق تقدم سريع، ولكنها صغيرة بما يكفي لتجنب القفز فوق الهدف.
- النتيجة : تصل إلى الحل الأمثل بكفاءة واستقرار.

الان لنرى تطبيق الدالة في بايثون :

```
86 def gradient_descent(f, init_x, lr=0.01, step_num=100):
87     x = init_x
88     x_history = []
89     for i in range(step_num):
90         x_history.append( x.copy() )
91         grad = numerical_gradient(f, x)
92         x -= lr * grad
93     return x, np.array(x_history)
```

الشرح:

نعرف دالة اسمها gradient_descent، ونحدد المدخلات:

- f (الدالة المراد تحسينها) : هذه هي الدالة التي نريد إيجاد قيمتها الدنيا (دالة الخسارة)
- init_x (القيمة الأولية) : هذه هي نقطة البداية. إنها عبارة عن مصفوفة تحتوي على القيم الأولية

لمعاملاتنا

(مثل np.array([-3.0, 4.0]))

- lr=0.01 (معامل التعلم) : هذا هو مقياس الخطوة الذي ناقشناه سابقاً. القيمة الافتراضية هنا هي 0.01، وهي قيمة شائعة. يمكنك تغيير هذه القيمة عند استدعاء الدالة إذا أردت تجربة خطوات أكبر أو أصغر.
- step_num=100 (عدد الخطوات) : هذا هو عدد التكرارات أو الحدود الأقصى لعدد الخطوات. القيمة الافتراضية هي 100 خطوة سيتوقف بعد ل 100

▪ $x = \text{init_x}$

نحن ننشئ نسخة من نقطة البداية ونسميها x سنقوم بتحديث هذا المتغير x في كل خطوة، بينما يبقى init_x كما هو.

▪ $x_history = []$

هذا سطر مهم جداً. نحن ننشئ قائمة فارغة. سنحفظ موقعه الحالي في هذه القائمة.

▪ حلقة التكرار for :

هذه هي الحلقة الرئيسية التي تتكرر step_num مرة (100 مرة افتراضياً). كل دورة في هذه الحلقة تمثل خطوة واحدة

▪ $x_history.append(x.copy())$

• تخيل معي أن المتغير x هو موقع المتسلق على الجبل في أي لحظة. والمتغير $x_history$ هو سجل الرحلة.

• نحن نريد أن نسجل كل خطوة يخطوها المتسلق لكي نتمكن لاحقاً من رسم مساره بالكامل على الخريطة ورؤية كيف وصل إلى القمة.

• لذلك، في كل مرة يتحرك فيها المتسلق، نأخذ صورة لموقعه الحالي ونلصقها في مذكرته ($x_history$). الدالة $\text{append}()$ هي التي تقوم بلصق هذه "الصورة" في نهاية القائمة.

▪ لماذا نستخدم $\text{copy}()$ ؟

عندما نتعامل مع كائنات متغيرة (Mutable Objects) مثل القوائم (Lists) أو مصفوفات (Arrays)، فإن المتغير لا يحتفظ بالقيمة مباشرة، بل يحتفظ ب مرجع أو مؤشر إلى مكان وجود القيمة في الذاكرة

▪ $\text{grad} = \text{numerical_gradient}(f, x)$

نستدعي دالة $\text{numerical_gradient}$ التي عرفناها سابقاً لحساب التدرج عند موقعنا الحالي x النتيجة grad هو متجه (سهم) يشير إلى اتجاه الصعود الأعلى

▪ $x -= \text{lr} * \text{grad}$: هذا هو تطبيق المعادلة

▪ $\text{lr} * \text{grad}$: نضرب متجه التدرج (اتجاه الصعود) في معامل التعلم (حجم الخطوة).

▪ $x -= \dots$: نطرح النتيجة من موقعنا الحالي x هذا يؤدي بنا إلى موقع جديد x يكون قد تحرك خطوة في الاتجاه المعاكس تماماً للتدرج (أي اتجاه الهبوط الأسرع).

▪ $\text{return } x, \text{np.array}(x_history)$

بعد أن تنتهي الحلقة التكرارية (بعد 100 خطوة)، تعود الدالة بقيمتين:

• x : هذا هو الموقع النهائي. إنه يمثل إحداثيات النقطة التي وجدتها الخوارزمية كأقل قيمة ممكنة للدالة بعد 100 خطوة.

• $\text{np.array}(x_history)$: هذا هو المسار الكامل، محولاً من قائمة إلى مصفوفة لتسهيل رسمه بيانياً

الخلاصة :

باختصار، هذه الدالة هي محاكاة لرحلة البحث عن الحل الأمثل. تأخذ خريطة (f)، ونقطة بداية (init_x)، وتحدد حجم الخطوات (lr) وعددها (step_num). ثم تبدأ رحلتها، مسجلة مسارها في كل مرة، وتحسب اتجاه الهبوط، وتتخذ خطوة، وفي النهاية تعلن عن وجهتها النهائية وتعرض لك خريطة رحلتها بالكامل.

الخوارزمية العملية خطوة بخطوة

يشير التدرج إلى الاتجاه الذي يزداد فيه الدالة بأسرع طريقة، وهو متجه يشير نحو اتجاه أعلى ارتفاع L حيث تزداد L (الخسارة) بأسرع ما يمكن. لذلك إذا أردنا تقليل L ، نتحرك في الاتجاه المعاكس (أكثر انخفاض حاد)

$$W \leftarrow W - \eta \nabla L$$

1- البداية العشوائية W Start at some random

• الشرح: بما أننا لا نعرف أين يقع أدنى وادٍ، فإننا نبدأ من مكان عشوائي على الجبل. نختار قيم أولية عشوائية للأوزان W .

2- حساب التدرج $\nabla W L$ Compute gradient

• الشرح: في موقعنا الحالي، نستخدم البوصلة لحساب التدرج. نعرف الآن اتجاه الصعود الأعلى.

3- التحرك في الاتجاه المعاكس Move a little opposite to it

• الشرح: نتخذ خطوة صغيرة (بحجم η في الاتجاه المعاكس تماماً للتدرج الذي حسبناه). هذا يقلل من قيمة الخسارة لائقربنا من الهدف.

4- التكرار "Repeat":

• الشرح: خطوة واحدة لا تكفي. نكرر العملية من البداية: من موقعنا الجديد، نحسب التدرج مرة أخرى، ونتخذ خطوة أخرى في الاتجاه المعاكس، وهكذا.

الهدف النهائي: الأمل في الوصول إلى القاع

بعد أن جهزنا خوارزمية البحث (دالة `gradient_descent`)، لنرى كيف نستخدم هذه الخوارزمية لحل مشكلة محددة

إعداد المهمة: نقطة البداية والخريطة

تخيل أنك مستكشف تبحث عن أدنى نقطة في وادٍ عميق وهي القيمة الدنيا لدالتنا (`function_2`) قبل أن تبدأ رحلتك، تحتاج إلى شيئين:

- خريطة المنطقة: هذه هي دالتنا `function_2`.
- نقطة بداية: هذا هو المكان الذي ستبدأ منه رحلتك

```
92 init_x = np.array([-3.0, 4.0])
93 grad = gradient_descent(function_2, init_x=init_x, lr=0.1, step_num=100)
```

الشرح:

▪ السطر الاول:

هنا اخترنا أن يبدأ مستكشفنا رحلته من الإحداثيات $(-3.0, 4.0)$ هذه نقطة عشوائية على "خريطتنا"، ليست قريبة من الهدف الهدف هو $(0, 0)$ كما نعلم

▪ السطر الثاني:

نحن نستدعي دالة `gradient_descent` ونمررها:

- function_2: الخريطة التي سيعمل عليها المستكشف.
- init_x=init_x: نقطة البداية التي حددناها.
- lr=0.1: حجم الخطوات التي سيخطوها المستكشف (معامل تعلم جيد ومناسب).
- step_num=100: الحد الأقصى لعدد الخطوات التي سيخطوها (100 خطوة كافية للوصول إلى الهدف في هذه الحالة)

النتيجة:

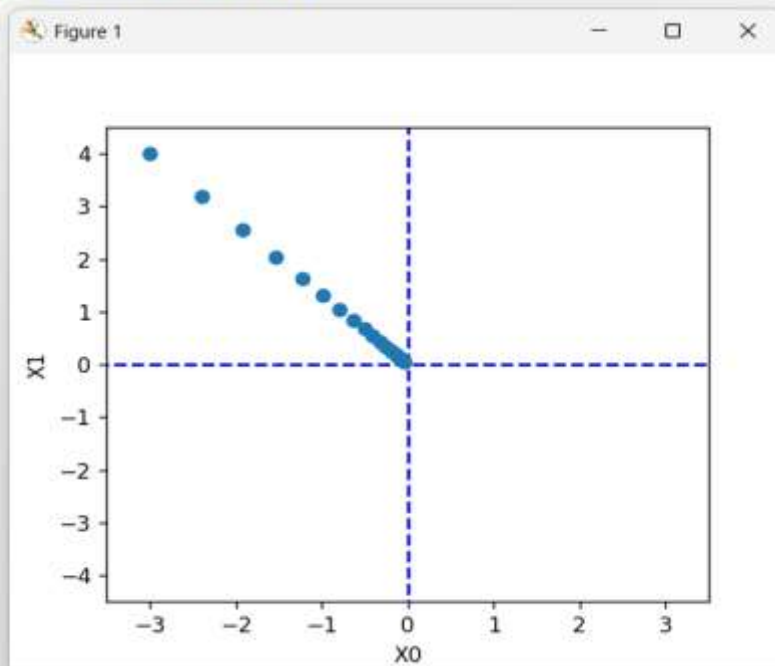
- بعد أن أكمل المستكشف رحلته المكونة من 100 خطوة، أعلن عن النقطة النهائية التي وصل إليها.
- النتيجة : $(-6.1e-10, 8.1e-10)$ رقم صغير جداً يقترب من الصفر.
- الخلاصة: المستكشف وصل إلى النقطة $(0, 0)$ تقريباً! وهذه هي القيمة الحقيقية الدنيا للدالة. هذا يعني أن خوارزمتنا نجحت في العثور على الهدف.

لنرى الرسم البياني :

```

81 init_x = np.array([-3.0, 4.0])
82 lr = 0.1
83 step_num = 20
84 x, x_history = gradient_descent(function_2, init_x, lr=lr, step_num=step_num)
85 plt.plot([-5, 5], [0,0], '--b')
86 plt.plot([0,0], [-5, 5], '--b')
87 plt.plot(x_history[:,0], x_history[:,1], 'o')
88 plt.xlim(-3.5, 3.5)
89 plt.ylim(-4.5, 4.5)
90 plt.xlabel("X0")
91 plt.ylabel("X1")
92 plt.show()
93

```



- 1- نقطة البداية عند $(-3, 4)$
- 2- سلسلة من النقاط (الدوائر) التي تشكل "مساراً" متعرجاً.
- 3- هذا المسار يتحرك بشكل واضح ومنهجي نحو المركز $(0, 0)$.
- 4- كل خطوة جديدة تكون أقرب إلى الهدف من سابقتها.

Gradients for a Neural Network التدرج في الشبكة العصبونية :

ننتقل من عالم الدوال البسيطة ذات المتغيرين إلى العالم الحقيقي للشبكات العصبونية، حيث المعاملات (Parameters) ليست مجرد أرقام فردية، بل هي مصفوفات (Matrices) كاملة من الأوزان (Weights)

$$\mathbf{W} = \begin{pmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \end{pmatrix}$$

$$\frac{\partial L}{\partial \mathbf{W}} = \begin{pmatrix} \frac{\partial L}{\partial w_{11}} & \frac{\partial L}{\partial w_{12}} & \frac{\partial L}{\partial w_{13}} \\ \frac{\partial L}{\partial w_{21}} & \frac{\partial L}{\partial w_{22}} & \frac{\partial L}{\partial w_{23}} \end{pmatrix}$$

التدرج ليس مجرد رقم، بل هو مصفوفة

لقد تعلمنا أن التدرج هو متجه يخبرنا بالاتجاه. لكن عندما تكون المعاملات التي نريد تحسينها هي مصفوفة أوزان $\mathbf{W}$ ، فإن التدرج أيضاً سيكون مصفوفة بنفس الحجم.

مصفوفة التدرج :

كل عنصر (رقم) داخل مصفوفة التدرج هذه يخبرنا بشيء محدد جداً إذا قمنا بتغيير هذا الوزن المحدد قليلاً، فكم ستتأثر دالة الخسارة $\mathcal{L}$ (أي خطأ الشبكة)؟

■ العنصر الأول : هذا هو الرقم الموجود في الصف الأول والعمود الأول من مصفوفة التدرج. وظيفته هي

قياس حساسية الوزن w_{11} الموجود في نفس الموقع في مصفوفة الأوزان $\mathbf{W}$

• إذا كانت قيمة $\frac{\partial L}{\partial w_{11}}$ كبيرة، فهذا يعني أن الوزن w_{11} حساس جداً. أي تغيير صغير فيه سيؤدي إلى تغيير كبير في خطأ الشبكة.

• إذا كانت قيمته قريبة من الصفر، فهذا يعني أن الوزن w_{11} غير حساس حالياً، وتغييره لن يؤثر كثيراً على الخطأ.

■ هذا ينطبق على كل عنصر في المصفوفة.

لماذا يجب أن يكون الشكل متطابقاً؟

لأنه إذا كانت مصفوفة الأوزان $\mathbf{W}$ تحتوي على 100 وزن مختلف، فنحن بحاجة إلى 100 قيمة مختلفة في مصفوفة التدرج لإخبارنا بكيفية تحديث كل واحد من هذه الأوزان المئة.

لتحقيق هذا التطابق المثالي، يجب أن تكون مصفوفة التدرج بنفس هيكل وتنظيم مصفوفة الأوزان

مثال عملي (2x3) :

■ إذا كانت مصفوفة الأوزان $\mathbf{W}$ بحجم (2x3)، فهذا يعني أن لدينا 6 أوزان :

($w_{11}, w_{12}, w_{13}, w_{21}, w_{22}, w_{23}$)

■ لذلك، يجب أن تكون مصفوفة التدرج أيضاً بحجم (2x3).

■ العنصر في الموقع (1,2) من مصفوفة التدرج سيعطينا تعليمات للوزن في الموقع (1,2) من مصفوفة الأوزان.

- العنصر في الموقع (2,3) من مصفوفة التدرج سيعطينا تعليمات للوزن في الموقع (2,3) من مصفوفة الأوزان

الآن، دعونا نبدأ في تنفيذ برنامج يحسب التدرج عن طريق أخذ شبكة عصبية بسيطة كمثال. للقيام بذلك، سنقوم بتنفيذ فئة باسم simpleNet :

هذا الكود يقوم بثلاثة أشياء رئيسية :

1. بناء دالة لحساب التدرج (numerical_gradient) تعمل على المصفوفات متعددة الأبعاد.
 2. تعريف الدوال الأساسية للشبكات العصبية مثل softmax و cross_entropy_error.
 3. تجميع كل شيء في فئة (Class) واحدة تمثل شبكة عصبية بسيطة (simpleNet).
- هذه النسخة من دالة numerical_gradient هي نسخة مطورة من السابقة.
- السابقة كانت تعمل بشكل جيد مع المتجهات (أي المصفوفات ذات البعد الواحد)، لكن في الشبكات العصبية الحقيقية، معاملاتنا (الأوزان) تكون على شكل مصفوفات متعددة الأبعاد (مثل 2x3 أو أكثر)

```

4 def numerical_gradient(f, x):
5     h = 1e-4
6     grad = np.zeros_like(x)
7     it = np.nditer(x, flags=['multi_index'], op_flags=['readwrite'])
8     while not it.finished:
9         idx = it.multi_index
10        tmp_val = x[idx]
11        x[idx] = tmp_val + h
12        fxh1 = f(x) # f(x+h)
13        x[idx] = tmp_val - h
14        fxh2 = f(x) # f(x-h)
15        grad[idx] = (fxh1 - fxh2) / (2*h)
16        x[idx] = tmp_val
17        it.iternext()
18    return grad

```

- grad = np.zeros_like(x) : كما قبل، ننشئ مصفوفة فارغة بنفس حجم وشكل المدخل x لتخزين النتائج.
- it = np.nditer(...) : هو مكرر ذكي (Iterator) وظيفته هي المرور على كل عنصر في المصفوفة x، بغض النظر عن عدد أبعادها (سواء كانت D1، D2، D3، أو أكثر).
- idx = it.multi_index : في كل مرة، يعطينا هذا المكرر فهرس (index) العنصر الحالي الذي يزوره. إذا كنا في الصف الثاني والعمود الأول، فسيكون idx هو (0, 1)
- حساب المشتق الجزئي : المنطق هو نفسه تماماً كما في الدالة السابقة. نأخذ العنصر الحالي x[idx]، ونحسب المشتق الجزئي للدالة f بالنسبة لهذا العنصر فقط.
- grad[idx] = ... : نحفظ نتيجة المشتق الجزئي في نفس الموقع في مصفوفة النتائج grad
- it.iternext() : نأمر المكرر بالانتقال إلى العنصر التالي في المصفوفة ونعيد العملية.

الخلاصة:

هذه الدالة الآن قادرة على حساب التدرج لأي دالة f بالنسبة لأي مدخل x، حتى لو كان x مصفوفة أوزان بحجم (100x50) ستقوم بحساب المشتقة الجزئية لكل واحد من الـ 5000 وزن وتخزينها في مصفوفة تدرج بنفس الحجم .

الدوال الأساسية للشبكات العصبية :

```

20 def softmax(x):
21     x = x-np.max(x, axis=-1, keepdims=True)
22     return np.exp(x) / np.sum(np.exp(x), axis=-1, keepdims=True)
23 def sigmoid(x):
24     return 1 / (1 + np.exp(-x))
25
26
27 def cross_entropy_error(y, t):
28     if y.ndim == 1:
29         t = t.reshape(1, t.size)
30         y = y.reshape(1, y.size)
31     if t.size == y.size:
32         t = t.argmax(axis=1)
33     batch_size = y.shape[0]
34     return -np.sum(np.log(y[np.arange(batch_size), t] + 1e-7)) / batch_size
--

```

▪ softmax(x) :

دالة تحول أي متجه من الأرقام إلى توزيع احتمالي. بمعنى آخر، تجعل جميع مخرجاتها موجبة ومجموعها يساوي 1. هذا مفيد جداً في مشاكل التصنيف، حيث يمكن تفسير المخرجات على أنها "احتمالات" لكل فئة.

▪ cross_entropy_error(y, t) :

دالة الخسارة (Loss Function). وظيفتها هي قياس مدى الخطأ في تنبؤات الشبكة. تقارن بين المخرجات المتوقعة من الشبكة (y) والمخرجات الحقيقية الصحيحة (t) كلما كانت قيمة الخسارة أصغر، كان أداء الشبكة أفضل.

[: class simpleNet](#)

```

39 class simpleNet:
40     def __init__(self):
41         self.W = np.random.randn(2,3)
42     def predict(self, x):
43         return np.dot(x, self.W)
44     def loss(self, x, t):
45         z = self.predict(x)
46         y = softmax(z)
47         loss = cross_entropy_error(y, t)
48         return loss
49 net = simpleNet()
50 print('E1 : ',net.W)
51 x = np.array([0.6, 0.9])
52 p = net.predict(x)
53 print('E2 : ',p)
54 print('E3 : ',np.argmax(p))
55 t = np.array([0, 0, 1])
56 print('E4 : ',net.loss(x, t))

```

PROBLEMS	OUTPUT	DEBUG CONSOLE	TERMINAL	PORTS
E1 :	[[-1.03945768 1.52282686 0.06905583] [-2.28322997 -0.13486398 -0.89913815]]			
E2 :	[-2.67858158 0.79231853 -0.76779083]			
E3 :	1			
E4 :	1.7761891354044688			

__init__(self) المُنشئ (Constructor) :

- هذه الدالة تُستدعى تلقائياً عند إنشاء كائن جديد من هذه الفئة
- self.W : هذا هو أهم متغير في الفئة. إنه يمثل مصفوفة الأوزان (Weights) ، وهي عقل الشبكة أو ذاكرتها. هذه هي القيم التي سيتعلمها النموذج ويحسنها أثناء التدريب.
- np.random.randn(2, 3) : هنا نقوم بتهيئة الأوزان بقيم عشوائية صغيرة مأخوذة من التوزيع الطبيعي (Gaussian distribution) هو (2x3) ، مما يعني أن الشبكة تتوقع مدخلات من حجم 2 وستعطي مخرجات من حجم 3.
- predict(self, x) - دالة التنبؤ (المرور الأمامي):
- الوظيفة : أخذ مدخل x وإنتاج تنبؤ.
- المنطق : هي ببساطة عملية ضرب المصفوفات (np.dot) نضرب مدخل الشبكة x في مصفوفة الأوزان self.W للحصول على المخرجات. هذا هو جوهر حساب الشبكة العصبونية من طبقة واحدة.

loss(self, x, t) - دالة الخسارة :

- الوظيفة : حساب مدى سوء أداء الشبكة.
- المدخلات :

- X : بيانات الإدخال .
- t : التسمية الصحيحة (the correct label) ، أي الإجابة الصحيحة التي نأمل أن تتنبأ بها الشبكة.
- المنطق خطوة بخطوة :

1. $z = \text{self.predict}(x)$: أولاً، قم بعملية التنبؤ للحصول على المخرجات الخام.
2. $y = \text{softmax}(z)$: ثانياً، حوّل هذه المخرجات الخام إلى احتمالات باستخدام دالة softmax
3. $\text{loss} = \text{cross_entropy_error}(y, t)$: ثالثاً، قارن هذه الاحتمالات المتوقعة بـ y بالإجابة الصحيحة t لحساب قيمة الخسارة النهائية.

الخلاصة النهائية:

الآن لدينا شبكة عصبونية كاملة (simpleNet) . يمكننا إنشاؤها، وإعطائها بيانات، وطلب منها أن تتنبأ. والأهم من ذلك، يمكننا أن نسألها: ما هي قيمة الخسارة لديك؟ . وباستخدام دالة numerical_gradient القوية التي بنيناها، يمكننا الآن حساب تدرج هذه الخسارة بالنسبة لأوزان الشبكة self.W، وهذا هو الخطوة الأولى والأهم في عملية "التدريب" أو "التعلم".

حساب تدرج أوزان الشبكة :

بعد أن أنشأنا شبكتنا العصبونية البسيطة simpleNet وأدواتنا الرياضية (numerical_gradient) ، الآن نريد الإجابة على السؤال الأهم

كيف يجب أن نغير أوزان الشبكة (net.W) لتحسين أدائها وتقليل الخطأ؟

الإجابة هي حساب التدرج. $dW = \frac{\partial(\text{loss})}{\partial(W)}$:

ولكن هناك مشكلة بسيطة في التوافق تحتاج إلى حل ذكي.

المشكلة: عدم التوافق بين الأدوات

- أداتنا: دالة `numerical_gradient(f, x)`. هذه الدالة مصممة لتعمل بطريقة معينة: تأخذ دالة (f) و متغيراً (x)، ثم تحسب مشتقة الدالة f بالنسبة للمتغير x.
 - حاجتنا: نريد حساب مشتقة دالة الخسارة `net.loss(x, t)` بالنسبة لمصفوفة الأوزان `net.W`. هل ترى المشكلة؟
 - الدالة التي نريد اشتقاقها هي `net.loss`.
 - المتغير الذي نريد الاشتقاق بالنسبة له هو `net.W`.
 - لكن `net.loss` لا تأخذ `net.W` كمُدخل مباشر! مدخلاتها هي x (بيانات) و t (تسميات صحيحة).
- لا يمكننا ببساطة كتابة `numerical_gradient(net.loss, net.W)` لأن `numerical_gradient` ستحاول استدعاء `net.loss(net.W)`، وهذا سيفشل لأن `net.loss` تتوقع مدخلات مختلفة.

الحل الذكي: "الدالة (Wrapper Function)"

لحل هذه المشكلة، نستخدم حيلة برمجية ذكية جداً وننشئ دالة وهمية (Wrapper/Dummy Function).

```
56 def f(W):
57     return net.loss(x, t)
```

الشرح :

- `def f(W)` : نحن نعرّف دالة جديدة اسمها f هذه الدالة تأخذ مدخلاً واحداً اسمه W
- `return net.loss(x, t)` : هنا تكمن الخدعة. الدالة ستتجاهل تماماً المدخل W الذي تم تمريره إليها. كل ما تفعله هو استدعاء دالة الخسارة الحقيقية `net.loss` بالمدخلات الصحيحة لها x و t
- لماذا هذا الحل يعمل؟ لأنه يخلق واجهة متوافقة مع دالة `numerical_gradient`

ربط الأجزاء معاً:

كيف تعمل العملية بأكملها؟

```
56 def f(W):
57     return net.loss(x, t)
58
59 dw = numerical_gradient(f, net.W)
60 print(dw)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
[[ 0.2575925  0.20315576 -0.46074825]
 [ 0.38638875  0.30473363 -0.69112238]]
```

Implementing a Training Algorithm

ننتقل من الدوال المنفصلة إلى بناء نموذج أو شبكة عصبية متكاملة داخل فئة (Class)

الفكرة الأساسية:

فكر في الفئة TwoLayerNet (Class) على أنها المخطط أو التصميم الهندسي لبناء مخ بسيط.

- عندما ننشئ كائناً (Object) من هذه الفئة، فإننا نحصل على مخ حقيقي وجاهز للعمل.
- هذا المخ لديه ذاكرة (params)
- وقدرة على التفكير (predict)
- وتقييم نفسه (loss)
- ووضع خطة للتحسين (numerical_gradient)

الشرح المفصل TwoLayerNet

2. __init__ لحظة "ولادة" الشبكة وتجهيز ذاكرتها

```
class TwoLayerNet:
    def __init__(self, input_size, hidden_size, output_size, weight_init_std=0.01):
        self.params = {}
        self.params['W1'] = weight_init_std * np.random.randn(input_size, hidden_size)
        self.params['b1'] = np.zeros(hidden_size)
        self.params['W2'] = weight_init_std * np.random.randn(hidden_size, output_size)
        self.params['b2'] = np.zeros(output_size)
```

هذه الدالة تستدعى تلقائياً عند إنشاء شبكة جديدة (مثلاً net = TwoLayerNet(...)) وتوظيفتها هي تهيئة ذاكرة الشبكة (

- : self.params = {}
- هنا ننشئ خزانة أو صندوق أمان (params) لتخزين كل معاملات الشبكة (الأوزان و الانحيازات)
- استخدمنا القاموس (Dictionary) ليسهل الوصول لكل معامل باسمه، مثل params['W1']
- self.params['W1'] = ...
- هذا هو "مصفوفة الأوزان للطبقة الأولى، هي التي تربط مدخلات الشبكة (الصور) بالطبقة المخفية.
- : np.random.randn(...)
- نبدأ بقيم عشوائية صغيرة. هذا مهم جداً لكسر "التماثل (Symmetry) " بين الخلايا العصبية، وإلا ستتعلم جميعها نفس الشيء.
- : input_size, hidden_size
- نحدد أبعاد المصفوفة بناءً على حجم المدخلات (784 بكسل) وحجم الطبقة المخفية (مثلاً 100 خلية).
- : self.params['b1'] = ...

متجه الانحياز للطبقة الأولى، هو قيمة واحدة تضاف لكل خلية في الطبقة المخفية. نبدأه بأصفار.

■ `self.params['W2']` و `self.params['b2']`

نفس الشيء، ولكن للطبقة الثانية التي تربط الطبقة المخفية بالمخرجات (10 أرقام).

الخلاصة : `init` يجهز بنية الشبكة الداخلية ويملأها بقيم أولية عشوائية للأوزان و صفرية للانحيازات

3. دالة التنبؤ `predict` : عملية "التفكير" أو المرور الأمامي

```
def predict(self, x):
    W1, W2 = self.params['W1'], self.params['W2']
    b1, b2 = self.params['b1'], self.params['b2']
    a1 = np.dot(x, W1) + b1
    z1 = sigmoid(a1)
    a2 = np.dot(z1, W2) + b2
    y = softmax(a2)
    return y
```

هذه هي الدالة التي تأخذ صورة ما وتتنبأ بالرقم الموجود فيها

■ `W1, W2 = ...`

نأخذ مصفوفتي الأوزان من "ذاكرة" الشبكة.

■ `a1 = np.dot(x, W1) + b1`

نضرب مدخلات الصورة في أوزان الطبقة الأولى ونضيف الانحياز. هذه هي العملية الحسابية الأولى.

■ `z1 = sigmoid(a1)`

نمرر النتيجة عبر دالة تنشيط (`sigmoid`) لإدخال "لاخطية" في الشبكة، مما يسمح لها بتعلم أنماط معقدة.

■ `a2 = np.dot(z1, W2) + b2`

نكرر العملية للطبقة الثانية.

■ `y = softmax(a2)`

نحول المخرجات النهائية إلى احتمالات. كل رقم يمثل احتمال أن تكون الصورة هي هذا الرقم.

■ `return y`

نرجع الاحتمالات كتنبؤ الشبكة.

4. دالة الخسارة `loss` تقييم الأداء :

```
def loss(self, x, t):
    y = self.predict(x)
    return cross_entropy_error(y, t)
```

هذه الدالة تقيس "مدى سوء" أداء الشبكة.

▪ : y = self.predict(x)

أولاً، نطلب من الشبكة أن تتنبأ (predict)

▪ :return cross_entropy_error(y, t)

ثانياً، نقارن تنبؤات الشبكة (y) بالإجابات الصحيحة (t) باستخدام دالة الخسارة (Cross-Entropy Error). كلما كانت النتيجة أصغر، كان أداء الشبكة أفضل.

5. دالة الدقة accuracy بطاقة التقييم:

```
def accuracy(self, x, t):
    y = self.predict(x)
    y = np.argmax(y, axis=1)
    t = np.argmax(t, axis=1)
    accuracy = np.sum(y == t) / float(x.shape[0])
    return accuracy
```

هذه الدالة تعطينا مقياساً أوضح وأكثر إنسانية: نسبة الإجابات الصحيحة.

▪ : y = self.predict(x)

نحصل على تنبؤات الشبكة (احتمالات).

▪ : y = np.argmax(y, axis=1)

نحوّل الاحتمالات إلى رقم واحد argmax يُرجع فهرس (index) أكبر احتمال. بما أن الاحتمالات مرتبة للأرقام 0-9، فإن هذا يعطينا التنبؤ النهائي (مثلاً، 5).

▪ : t = np.argmax(t, axis=1)

نفعل نفس الشيء للإجابات الصحيحة t لأنها بصيغة one-hot

▪ : accuracy = np.sum(y == t) / ...

نقارن التنبؤات بالإجابات الصحيحة

نحسب عدد التطابقات

ونقسم على العدد الإجمالي للحصول على نسبة مئوية

```
def numerical_gradient(self, x, t):
    loss_W = lambda W: self.loss(x, t)
    grads = {}
    grads['W1'] = numerical_gradient(loss_W, self.params['W1'])
    grads['b1'] = numerical_gradient(loss_W, self.params['b1'])
    grads['W2'] = numerical_gradient(loss_W, self.params['W2'])
    grads['b2'] = numerical_gradient(loss_W, self.params['b2'])
    return grads
```

هذه هي الدالة الأهم. هي لا "تعلّم" الشبكة، بل تقوم بحساب خطة التحسين.

- : loss_W = lambda W: self.loss(x, t)
- نحن ننشئ دالة وهمية (lambda) لا تهتم بالمدخل W ، بل تستدعي دالة الخسارة للشبكة بأكملها.
- : grads = {}
- ننشئ قاموساً جديداً لتخزين خطة التحسين
- : grads['W1'] = numerical_gradient(loss_W, self.params['W1'])
- نستدعي دالة التدرج العددي العامة (numerical_gradient) ونطلب منها:
- 1. خذ مصفوفة الأوزان. self.params['W1']
- 2. عدّلها قليلاً (زيادة ونقصان).
- 3. في كل مرة، استخدم الدالة loss_W لحساب الخسارة.
- 4. أعطينا خطة التحسين (التدرج) لهذه المصفوفة.
- 5. نكرر هذا لكل معامل (W1, b1, W2, b2) ونخزن الخطط في القاموس grads

الآن بعد أن تعرفنا على الدوال التي بداخل الكلاس حان وقت التنفيذ لتوضيح كيفية الاستخدام :

```
79 net = TwoLayerNet(input_size=784, hidden_size=100, output_size=10)
80 print('Shape of W1 : ',net.params['W1'].shape)
81 print('Shape of B1 : ',net.params['b1'].shape)
82 print('Shape of W2 : ',net.params['W2'].shape)
83 print('Shape of B2 : ',net.params['b2'].shape)
```

```
PROBLEMS 13 OUTPUT DEBUG CONSOLE TERMINAL PORTS
Shape of W1 : (784, 100)
Shape of B1 : (100,)
Shape of W2 : (100, 10)
Shape of B2 : (10,)
```

الشرح :

- السطر الاول : net = TwoLayerNet(input_size=784, hidden_size=100, output_size=10) إنشاء كائن (Object) من الشبكة العصبية

▪ `input_size=784` :

المدخلات ستكون على شكل متجه من 784 عنصراً لأن صور MNIST بحجم 28×28 بكسل، و $784 = 28 \times 28$

▪ `hidden_size=100`:

نحن نحدد حجم الطبقة المخفية، وهي مكان "التفكير" في المخ. اخترنا أن يكون فيها 100 خلية عصبية.

▪ `output_size=10`:

المخرجات يجب أن تكون 10 قيم (تمثل الأرقام من 0 إلى 9)

▪ `net.params['W1'].shape` السطر الثاني :

سيعطيك (784,100) مصفوفة الربط بين 784 مدخلاً و 100 خلية مخفية.

▪ `net.params['b1'].shape` السطر الثالث :

سيعطيك (100,) متجه التحيز الخاص بالـ 100 خلية المخفية.

ونفس الشيء لبقية التعليمات

عملية التنبؤ:

```
x = np.random.rand(100, 784)
y = net.predict(x)
```

▪ `x = np.random.rand(100, 784)` :

هذا السطر ينشئ بيانات وهمية (Dummy Data) لقد أنشأنا 100 صورة وهمية، كل صورة هي متجه من 784 قيمة عشوائية.

▪ `y = net.predict(x)` :

نحن نستدعي دالة `predict` لنمرر لها الصور العشوائية `x` فنقوم باستخدام ذاكرتها (`net.params`) لمعالجة هذه الصور وتعطينا تنبؤاتها `y`

▪ النتيجة `y` ستكون مصفوفة بحجم (100, 10)، حيث كل صف يحتوي على 10 احتمالات (احتمال أن تكون الصورة رقم 0، 1، 2، ... إلخ).

```

85 x = np.random.rand(100, 784)
86 t = np.random.rand(100, 10)
87 grads = net.numerical_gradient(x, t)
88 print('Shape of W1 :', grads['W1'].shape )
89 print('Shape of B1 :', grads['b1'].shape )
90 print('Shape of W2 :', grads['W2'].shape )
91 print('Shape of B2 :', grads['b2'].shape )

```

PROBLEMS 13

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```

Shape of W1 : (784, 100)
Shape of B1 : (100,)
Shape of W2 : (100, 10)
Shape of B2 : (10,)

```

■ `t = np.random.rand(100, 10)`

هذا السطر ينشئ تسميات صحيحة وهمية للصور الـ 100 التي أنشأناها. كل تسمية هي متجه من 10 قيم صيغة one-hot

■ `grads = net.numerical_gradient(x, t)`

هذا هو قلب عملية التعلم.

نحن نستدعي دالة `numerical_gradient` نعطيها الصور `x` والتسميات الصحيحة `t`

تقوم هذه الدالة بحساب التدرجات (Gradients)، وهي خريطة تعليمات تخبر المخ كيف يجب أن يعدل "ذاكرته" (`params`) ليصبح أفضل في المرة القادمة.

■ `grads['W1'].shape`

هذا السطر يفحص شكل خطة التحسين الخاصة بأوزان الطبقة الأولى حيث سيكون الناتج (784, 100) ونفس الشيء لبقية التعليمات